

Catatan Ujian Akhir Semester

IF2230 Jaringan Komputer

Semester 4 Tahun 2025/2026



Narendra Dharma Wistara M.

13524044

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

June 6, 2026

Daftar Isi

1	Informasi Ujian	1
2	Capaian Pembelajaran Mata Kuliah (CPMK)	1
2.1	Tabel Keterpenuhan CPMK	1
3	Daftar Materi yang Diujikan	1
4	Outline Keseluruhan Materi	1
4.1	Transport Layer	1
4.2	Congestion Control	2
4.3	Application Layer	2
4.4	Wireless Network	2
4.5	Network Security	2
5	Transport Layer	3
5.1	Outline Fundamental Konsep Materi	3
5.2	Penjelasan Konsep	4
5.2.1	Pengantar Transport Layer	4
5.2.2	UDP	5
5.2.3	TCP	6
5.2.4	TCP Reliability dan Sliding Window	6
5.2.5	Adaptive Retransmission	7
5.2.6	Flow Control	8
5.2.7	Triggering Transmission dan Kinerja TCP	8
5.2.8	RPC	9
5.2.9	RTP dan RTCP	9
5.2.10	SCTP	10
5.2.11	QUIC sebagai Transport Modern Berbasis UDP [†]	10
5.2.12	Relasi Lintas Materi	11
5.3	Algoritma dan Rumus Penting	11
5.3.1	UDP Checksum	11
5.3.2	TCP Four-Tuple Demultiplexing	12
5.3.3	Adaptive RTT Estimation	12
5.3.4	Flow Control Window	12
5.3.5	MSS dan Delay-Bandwidth Product	13
5.3.6	Nagle's Algorithm	13
6	Congestion Control	14
6.1	Outline Fundamental Konsep Materi	14
6.2	Penjelasan Konsep	15
6.2.1	Konsep Dasar Alokasi Sumber Daya dan Congestion	15
6.2.2	Kriteria Evaluasi	16
6.2.3	Disiplin Antrean Router	17
6.2.4	TCP Congestion Control	18
6.2.5	Congestion Avoidance Lanjutan	19
6.2.6	Quality of Service	20
6.2.7	Relasi Lintas Materi	21
6.3	Algoritma dan Rumus Penting	21
6.3.1	Jain's Fairness Index	21
6.3.2	Fair Queuing Finish Time	22
6.3.3	AIMD dan Slow Start	22
6.3.4	RED Average Queue Length dan Drop Probability	22
6.3.5	Token Bucket	23

7	Application Layer	24
7.1	Outline Fundamental Konsep Materi	24
7.2	Penjelasan Konsep	25
7.2.1	Fondasi Application Layer	25
7.2.2	Domain Name System (DNS)	26
7.2.3	Web dan HTTP	27
7.2.4	Electronic Mail	28
7.2.5	Arsitektur Lanjutan	29
7.2.6	Protokol Aplikasi Khusus	30
7.2.7	Relasi Lintas Materi	30
7.3	Algoritma dan Rumus Penting	31
7.3.1	DNS Resource Record	31
7.3.2	Iterative DNS Resolution	31
7.3.3	Non-persistent HTTP Response Time	32
7.3.4	Conditional GET	32
7.3.5	Base64 Encoding Size	32
8	Wireless Network	33
8.1	Outline Fundamental Konsep Materi	33
8.2	Penjelasan Konsep	33
8.2.1	Fondasi dan Elemen Wireless Network	33
8.2.2	Standar dan Karakteristik Link	34
8.2.3	MAC 802.11	35
8.2.4	Mobilitas dan Routing	36
8.2.5	Wireless Security	37
8.2.6	Relasi Lintas Materi	38
8.3	Algoritma dan Rumus Penting	38
8.3.1	CSMA/CA dengan RTS/CTS	38
8.3.2	Mobile IP Tunneling	39
8.3.3	Delay Spread Intuition pada Multipath	39
9	Network Security	40
9.1	Outline Fundamental Konsep Materi	40
9.2	Penjelasan Konsep	41
9.2.1	Fondasi Network Security	41
9.2.2	Kerentanan per Layer	41
9.2.3	Cryptographic Building Blocks	42
9.2.4	Key Predistribution, PKI, dan Authentication Protocols	43
9.2.5	Security Systems by Layer	43
9.2.6	Firewalls, Filtering, Malware, dan Availability Attacks	44
9.2.7	Relasi Lintas Materi	45
9.3	Algoritma dan Rumus Penting	45
9.3.1	Stream Cipher XOR	45
9.3.2	Cipher Block Chaining	46
9.3.3	HMAC Konseptual	46
9.3.4	Challenge-Response dengan Nonce	46
9.3.5	SYN Flood Intuition	47

1 Informasi Ujian

Jenis Ujian	UAS
Nama Mata Kuliah	IF2230 Jaringan Komputer
Tanggal Ujian	Senin, 8 Juni 2026

Tabel 1: Informasi ujian

2 Capaian Pembelajaran Mata Kuliah (CPMK)

1. CPMK1. Menyebutkan struktur tipikal layer arsitektur network
2. CPMK2. Menjelaskan prinsip dasar medium access layer
3. CPMK3. Menjelaskan prinsip switched networks dan reliable message delivery
4. CPMK4. Menjelaskan prinsip routing dan packet forwarding pada jaringan ip
5. CPMK5. Mengkonfigurasi lan dan jaringan ip
6. CPMK6. Mengimplementasikan aplikasi jaringan/aplikasi client server sederhana

2.1 Tabel Keterpenuhan CPMK

CPMK	Transport	Congestion	Application	Wireless	Security
CPMK1	✓	✓	✓	✓	✓
CPMK2				✓	
CPMK3	✓	✓			✓
CPMK4		✓		✓	✓
CPMK5			✓	✓	✓
CPMK6	✓		✓		✓

Tabel 2: Keterpenuhan CPMK per materi yang dibahas

3 Daftar Materi yang Diujikan

1. Transport Layer
2. Congestion Control
3. Application Layer
4. Wireless Network
5. Network Security

4 Outline Keseluruhan Materi

Bagian ini menyajikan peta belajar ringkas untuk seluruh materi yang diujikan. Penjelasan mendalam tersedia pada section masing-masing materi yang sudah disusun.

4.1 Transport Layer

- Komunikasi logis process-to-process di atas layanan host-to-host IP.
- UDP sebagai simple demultiplexer dan TCP sebagai reliable byte stream.
- Socket API, port, segment, checksum, sequence number, ACK, sliding window, flow control, dan adaptive retransmission.
- RPC, RTP/RTCP, SCTP, TCP high-speed extensions, QUIC sebagai tambahan pengetahuan umum, serta relasi dengan aplikasi, congestion, wireless, dan security.

4.2 Congestion Control

- Resource allocation, bottleneck, congestion collapse, fairness, dan power.
- FIFO, tail drop, Fair Queuing, Weighted Fair Queuing, AIMD, Slow Start, Fast Retransmit, Fast Recovery, Tahoe, dan Reno.
- DECbit, RED, TCP Vegas, ECN, QoS, IntServ/RSVP, DiffServ, dan equation-based congestion control.

4.3 Application Layer

- Arsitektur client-server, peer-to-peer, hybrid, socket, port, dan kebutuhan layanan aplikasi.
- DNS, HTTP, email, FTP, SNMP, CDN, overlay, web services, P2P, dan multimedia signaling.
- Relasi aplikasi dengan TCP/UDP, DNS MX, HTTP persistent connection, TLS, dan routing tingkat aplikasi.

4.4 Wireless Network

- Wireless host, base station, wireless link, infrastructure/ad hoc mode, handoff, dan taksonomi single/multi-hop.
- WLAN/Wi-Fi, Bluetooth, 802.11, cellular, path loss, interference, multipath, CSMA/CA, RTS/CTS, association, dan mobility.
- Wireless security, TCP over wireless, Mobile IP, MANET routing, dan fitur efisiensi 802.11ax.

4.5 Network Security

- Authentication, confidentiality, integrity, availability, dan model ancaman Alice-Bob-Mallory.
- Kerentanan IP/TCP/aplikasi, symmetric cipher, public-key cipher, hash, MAC, PKI, certificate, nonce, challenge-response, Diffie-Hellman, Kerberos.
- PGP, SSH, TLS, IPsec, wireless security, firewall, NAT, filtering, malware, botnet, DDoS, dan DNSSEC.

5 Transport Layer

Transport Layer adalah lapisan yang membuat aplikasi dapat berkomunikasi sebagai proses, bukan sekadar sebagai host. Fokus ujian pada materi ini adalah memahami perbedaan UDP dan TCP, bagaimana TCP membangun reliability di atas IP best-effort, serta kapan mekanisme seperti sliding window, timeout, flow control, RPC, RTP, dan ekstensi TCP diperlukan.

Materi ini menjadi jembatan langsung ke Application Layer dan Congestion Control. Aplikasi memilih layanan transport sesuai kebutuhan data loss, delay, dan bandwidth; sementara TCP menggabungkan reliability, flow control, dan sinyal awal congestion control dalam satu mekanisme end-to-end.

5.1 Outline Fundamental Konsep Materi

1. **WAJIB** Pengantar Transport Layer
 - (a) **WAJIB** Komunikasi host-to-host dan process-to-process
 - (b) **WAJIB** Layanan IP best-effort dan masalah loss, reorder, duplicate, delay
 - (c) **WAJIB** Socket API, port number, multiplexing, dan demultiplexing
2. **WAJIB** UDP
 - (a) **WAJIB** Connectionless unreliable datagram
 - (b) **WAJIB** Header UDP, checksum, dan pseudoheader
 - (c) **PAHAM** Penggunaan UDP pada DNS, TFTP, SNMP, VoIP, dan multimedia
3. **WAJIB** TCP
 - (a) **WAJIB** Reliable, connection-oriented, full-duplex byte stream
 - (b) **WAJIB** Segment format, sequence number, ACK number, advertised window, checksum, dan flags
 - (c) **WAJIB** Three-way handshake, teardown, dan state TCP
4. **WAJIB** TCP Reliability dan Sliding Window
 - (a) **WAJIB** Sequence number, cumulative ACK, duplicate ACK, timeout, dan retransmission
 - (b) **WAJIB** Sender/receiver buffer pointer
 - (c) **WAJIB** Adaptive retransmission: EstimatedRTT, Karn/Partridge, Jacobson/Karels
5. **WAJIB** Flow Control
 - (a) **WAJIB** Receiver buffer, advertised window, dan effective window
 - (b) **PAHAM** Perbedaan flow control dan congestion control
6. **PAHAM** Triggering Transmission dan Kinerja TCP
 - (a) **WAJIB** MSS, MTU, delay-bandwidth product, dan keeping the pipe full
 - (b) **PAHAM** Nagle's Algorithm dan Silly Window Syndrome
 - (c) **PAHAM** Window scaling, timestamp, dan PAWS
7. **PAHAM** RPC
 - (a) **PAHAM** Request/reply abstraction
 - (b) **PAHAM** Marshalling, XDR/NDR, dan stub compiler
8. **PAHAM** RTP dan RTCP
 - (a) **PAHAM** Real-time transport di atas UDP
 - (b) **PAHAM** Application Level Framing, timestamp, sequence number, dan feedback RTCP

9. **PAHAM** SCTP
10. **PAHAM** QUIC sebagai transport modern berbasis UDP[†]
11. **WAJIB** Relasi Transport Layer dengan Application Layer, Congestion Control, Wireless Network, dan Network Security

5.2 Penjelasan Konsep

5.2.1 Pengantar Transport Layer

Inti Konsep.

Transport Layer menyediakan komunikasi logis antar-proses aplikasi yang berjalan pada host berbeda. Lapisan ini penting karena IP hanya mengirim paket antar-host secara best-effort, sedangkan aplikasi membutuhkan abstraksi yang mengenal proses, port, reliability, urutan data, dan kontrol aliran.

Motivasi utama Transport Layer adalah kesenjangan antara kebutuhan aplikasi dan layanan Network Layer. **Network Layer** hanya mengetahui alamat host dan rute paket; ia tidak mengetahui proses aplikasi mana yang harus menerima data. Selain itu, layanan IP bersifat **best-effort**, yaitu paket dapat hilang, terlambat, terduplikasi, datang tidak berurutan, atau terlalu besar sehingga perlu dipecah.

Definisi formalnya: **Transport Layer** adalah lapisan end-to-end yang mengubah layanan **host-to-host delivery** menjadi **process-to-process communication**. Protokol ini berjalan pada end systems, bukan pada router inti. Karena itu, fungsi seperti reliability, flow control, dan sebagian besar logika koneksi ditempatkan pada host pengirim dan penerima.

Mekanisme dasarnya berjalan sebagai berikut:

1. Proses aplikasi menulis data ke **socket**, yaitu API yang menjadi pintu antara aplikasi dan sistem transport milik sistem operasi.
2. Transport Layer menambahkan informasi endpoint, terutama **port number**, agar data dapat diarahkan ke proses tujuan.
3. Pada sisi pengirim, **multiplexing** menggabungkan data dari banyak proses aplikasi ke layanan jaringan yang sama.
4. Pada sisi penerima, **demultiplexing** membaca field port dan alamat endpoint untuk menyerahkan segmen ke socket yang tepat.

Komponen utama pada pengantar Transport Layer adalah sebagai berikut.

Komponen / Istilah	Definisi atau Peran
Host-to-host	Komunikasi antar-mesin berdasarkan alamat IP; disediakan oleh Network Layer.
Process-to-process	Komunikasi antar-proses aplikasi; disediakan oleh Transport Layer melalui port dan socket.
Socket	Antarmuka pemrograman yang digunakan aplikasi untuk mengirim dan menerima data jaringan.
Port number	Bilangan logis yang mengidentifikasi proses atau layanan pada host.
Multiplexing	Penggabungan data dari banyak proses aplikasi ke satu layanan jaringan.
Demultiplexing	Pemisahan segmen masuk ke proses aplikasi yang benar berdasarkan endpoint.

Tabel 3: Komponen dasar Transport Layer

Pada UDP, demultiplexing terutama menggunakan port tujuan. Pada TCP, koneksi dibedakan dengan 4-tuple:

$$\langle \text{SrcPort}, \text{SrcIPAddr}, \text{DstPort}, \text{DstIPAddr} \rangle$$

Keempat komponen ini memungkinkan satu server web pada port 80 melayani banyak koneksi klien secara bersamaan tanpa aliran datanya tertukar.

Konsep ini menjadi prasyarat semua subtopik lain. UDP adalah bentuk minimal dari process-to-process delivery, sedangkan TCP memperkaya abstraksi tersebut dengan reliability, byte stream, flow control, dan connection management.

5.2.2 UDP

Inti Konsep.

UDP adalah protokol transport sederhana yang menyediakan layanan datagram connectionless dan unreliable. UDP digunakan ketika aplikasi membutuhkan overhead rendah dan delay kecil, serta masih dapat menoleransi sebagian packet loss atau mengatur reliability sendiri di Application Layer.

UDP ada karena tidak semua aplikasi membutuhkan layanan berat seperti TCP. Aplikasi real-time seperti VoIP atau streaming lebih terganggu oleh retransmission delay daripada oleh hilangnya sebagian kecil data. DNS dan SNMP juga membutuhkan pertukaran pesan pendek yang cepat, sehingga connection setup TCP sering kali terlalu mahal.

Secara formal, **User Datagram Protocol (UDP)** adalah **simple demultiplexer**: ia memperluas layanan host-to-host IP menjadi process-to-process dengan menambahkan port, panjang datagram, dan checksum. UDP tidak membuat koneksi, tidak menyimpan state koneksi, tidak memakai sequence number, dan tidak memberi ACK bawaan.

Mekanisme UDP:

1. Aplikasi menulis satu pesan ke socket UDP.
2. UDP membentuk datagram dengan header berisi source port, destination port, length, dan checksum.
3. Datagram diserahkan ke IP tanpa handshake.
4. Host tujuan memakai destination port untuk menyerahkan datagram ke socket aplikasi.
5. Jika tidak ada proses yang mendengarkan port tersebut, host dapat mengirim ICMP Port Unreachable.

Field UDP	Peran
Source port	Port aplikasi pengirim; dapat dipakai penerima untuk membalas.
Destination port	Port aplikasi tujuan; menjadi dasar demultiplexing di host penerima.
Length	Panjang total UDP header dan data.
Checksum	Deteksi error bit pada UDP datagram, termasuk pseudoheader IP.

Tabel 4: Field header UDP

Checksum UDP dihitung dengan 1's complement sum atas word 16-bit dari UDP header, data, dan **pseudoheader**. Pseudoheader bukan field UDP yang dikirim sebagai header nyata, tetapi data dari IP header seperti source IP, destination IP, protocol number, dan UDP length yang ikut dihitung untuk mendeteksi misdelivery endpoint.

UDP cocok untuk DNS, TFTP, SNMP, RIP, VoIP, online gaming, dan multimedia. Bila aplikasi seperti TFTP tetap membutuhkan reliability, ia harus membangun ACK, timeout, dan retransmission sendiri pada Application Layer. Perbandingan ini memperjelas batas UDP dan TCP: UDP memprioritaskan kesederhanaan dan waktu, TCP memprioritaskan keandalan.

5.2.3 TCP

Inti Konsep.

TCP adalah protokol transport connection-oriented yang menyediakan reliable full-duplex byte stream. TCP penting karena banyak aplikasi seperti HTTP, SMTP, dan FTP membutuhkan data yang utuh, berurutan, tidak terduplikasi, dan dapat dibaca sebagai aliran byte yang stabil.

TCP lahir dari masalah yang sama dengan UDP, tetapi mengambil keputusan desain yang berbeda. Jika UDP menyerahkan hampir semua risiko ke aplikasi, TCP mengambil tanggung jawab reliability di Transport Layer. Dengan demikian, aplikasi dapat membaca dan menulis byte tanpa terus-menerus memikirkan packet loss, reorder, duplicate, dan flow mismatch.

****Reliable byte stream**** berarti TCP tidak mempertahankan batas pesan aplikasi. Jika aplikasi menulis 100 byte lalu 200 byte, penerima bisa saja membaca 300 byte sebagai satu aliran. Bila aplikasi membutuhkan batas record, aplikasi harus menambahkan panjang pesan, delimiter, atau format framing sendiri.

TCP memakai header yang lebih kaya daripada UDP.

Field / Flag TCP	Definisi atau Peran
Sequence number	Nomor byte pertama pada segmen; dasar pengurutan dan retransmission.
Acknowledgment number	Byte berikutnya yang diharapkan penerima; ACK bersifat kumulatif.
Advertised window	Ruang buffer yang tersedia pada receiver; dasar flow control.
Checksum	Deteksi error pada TCP segment dan pseudoheader IP.
SYN	Sinkronisasi initial sequence number saat membuka koneksi.
FIN	Menandakan satu arah aliran sudah selesai mengirim data.
ACK	Menandakan field acknowledgment valid.
RST	Membatalkan koneksi secara paksa.
PSH	Meminta penerima segera mendorong data ke aplikasi.
URG	Menandakan adanya urgent data yang ditunjuk urgent pointer.

Tabel 5: Field dan flag penting TCP

Three-way handshake:

1. Client mengirim SYN dengan initial sequence number x .
2. Server membalas SYN+ACK dengan initial sequence number y dan acknowledgment $x + 1$.
3. Client mengirim ACK dengan acknowledgment $y + 1$; setelah ini data dapat mengalir.

Teardown memakai FIN secara independen untuk setiap arah. Pihak yang selesai mengirim data mengirim FIN; lawan membalas ACK. Jika lawan juga selesai, ia mengirim FIN dan menerima ACK terakhir. State penting yang perlu dikenali adalah LISTEN, SYN_SENT, SYN_RCVD, ESTABLISHED, FIN_WAIT_1, FIN_WAIT_2, CLOSE_WAIT, LAST_ACK, TIME_WAIT, dan CLOSED.

TCP berkaitan erat dengan Application Layer karena memberi abstraksi socket yang nyaman untuk aplikasi intoleran data loss. TCP juga menjadi dasar Congestion Control karena loss dan duplicate ACK dipakai sebagai sinyal jaringan.

5.2.4 TCP Reliability dan Sliding Window

Inti Konsep.

Reliability TCP dibangun dengan sequence number, cumulative ACK, retransmission, dan sliding window. Sliding window memungkinkan banyak byte berada in flight sekaligus, sehingga koneksi tetap efisien tanpa kehilangan kemampuan mengurutkan dan mengirim ulang data.

Motivasi reliability adalah IP dapat menjatuhkan, menduplikasi, atau mengacak urutan paket. Jika aplikasi harus menangani semua ini sendiri, setiap aplikasi akan mengulang mekanisme yang sama. TCP memusatkan mekanisme tersebut pada Transport Layer.

****Sequence number**** memori byte, bukan sekadar paket. ****Acknowledgment number**** menyatakan byte berikutnya yang diharapkan penerima. Karena ACK TCP kumulatif, ACK 120 berarti semua byte sampai 119 sudah diterima berurutan. Jika ACK untuk byte sebelumnya hilang, ACK yang lebih besar tetap cukup untuk mengonfirmasi data sebelumnya.

Sliding window menjaga pointer berikut.

Pointer	Makna
LastByteAked	Byte terakhir yang sudah diakui penerima.
LastByteSent	Byte terakhir yang sudah dikirim ke jaringan.
LastByteWritten	Byte terakhir yang ditulis aplikasi ke send buffer.
LastByteRead	Byte terakhir yang sudah dibaca aplikasi penerima.
NextByteExpected	Byte berikutnya yang diharapkan agar urutan tetap kontigu.
LastByteRcvd	Byte terakhir yang sudah masuk receive buffer.

Tabel 6: Pointer sliding window TCP

Kondisi dasar sender:

$$\text{LastByteAked} \leq \text{LastByteSent} \leq \text{LastByteWritten}$$

Kondisi dasar receiver:

$$\text{LastByteRead} < \text{NextByteExpected} \leq \text{LastByteRcvd} + 1$$

Retransmission dipicu oleh timeout atau oleh indikasi kuat bahwa paket hilang. ****Duplicate ACK**** muncul ketika penerima terus mengharapkan byte yang sama karena ada lubang dalam urutan byte. Tiga duplicate ACK lazim dipakai oleh Fast Retransmit untuk mengirim ulang segmen yang hilang tanpa menunggu timeout penuh.

Bagian ini menjadi fondasi Congestion Control. Timeout dan duplicate ACK tidak hanya menandakan data perlu dikirim ulang, tetapi juga sering dipakai TCP sebagai sinyal bahwa jaringan mungkin macet.

5.2.5 Adaptive Retransmission

Inti Konsep.

Adaptive retransmission menentukan timeout TCP berdasarkan pengukuran RTT yang berubah-ubah. Konsep ini penting karena timeout yang terlalu pendek menyebabkan retransmission palsu, sedangkan timeout yang terlalu panjang membuat pemulihan loss menjadi lambat.

RTT Internet tidak konstan karena antrean router, rute, beban jaringan, dan karakteristik link dapat berubah. Karena itu, TCP tidak memakai timeout tetap. TCP mengukur ****SampleRTT****, memperbarui ****EstimatedRTT****, lalu menentukan ****Timeout****.

Algoritma awal menggunakan exponential weighted moving average:

$$\text{EstimatedRTT} = \alpha \times \text{EstimatedRTT} + (1 - \alpha) \times \text{SampleRTT}$$

dengan α biasanya sekitar 0.8 sampai 0.9. Timeout sederhana:

$$\text{Timeout} = 2 \times \text{EstimatedRTT}$$

Masalahnya adalah ****acknowledgment ambiguity****: jika segmen dikirim ulang, ACK yang datang bisa saja untuk transmisi pertama atau retransmission. ****Karn/Partridge Algorithm**** mengatasi ini dengan dua aturan: jangan mengambil SampleRTT dari paket retransmitted, dan gandakan timeout setelah retransmission sebagai exponential backoff.

****Jacobson/Karels Algorithm**** menambahkan deviasi RTT:

$$\text{Difference} = \text{SampleRTT} - \text{EstimatedRTT}$$

$$\text{EstimatedRTT} = \text{EstimatedRTT} + \delta \times \text{Difference}$$

$$\text{Deviation} = \text{Deviation} + \delta \times (|\text{Difference}| - \text{Deviation})$$

$$\text{TimeOut} = \mu \times \text{EstimatedRTT} + \phi \times \text{Deviation}$$

Sumber menyebut nilai praktis $\mu = 1$ dan $\phi = 4$. Jika variasi RTT kecil, timeout dekat dengan EstimatedRTT; jika variasi besar, deviasi memperbesar timeout agar TCP tidak terlalu agresif.

Adaptive retransmission berkaitan langsung dengan sliding window dan congestion control. Salah estimasi timeout dapat membuat TCP mengirim ulang data yang sebenarnya belum hilang, lalu memperparah beban jaringan.

5.2.6 Flow Control

Inti Konsep.

Flow control membatasi pengirim agar tidak menenggelamkan buffer penerima. Mekanisme ini menggunakan advertised window dari receiver, sehingga jumlah byte in flight tidak melebihi ruang yang masih tersedia pada receive buffer.

Motivasi flow control berbeda dari congestion control. ****Flow control**** melindungi receiver yang lambat atau buffer-nya hampir penuh. ****Congestion control**** melindungi router dan link di tengah jaringan. Keduanya sama-sama membatasi laju pengiriman, tetapi objek yang dilindungi berbeda.

Receiver menghitung ruang buffer tersisa dan mengiklankannya melalui field ****AdvertisedWindow**** pada header TCP. Rumus ruang receive buffer:

$$\text{RcvWindow} = \text{MaxRcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

Versi yang memakai NextByteExpected:

$$\text{AdvertisedWindow} = \text{MaxRcvBuffer} - ((\text{NextByteExpected} - 1) - \text{LastByteRead})$$

Sender wajib memenuhi:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{AdvertisedWindow}$$

Ruang efektif untuk pengiriman baru:

$$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

Jika EffectiveWindow bernilai nol, sender tidak boleh mengirim data baru sampai receiver mengiklankan ruang baru.

Flow control adalah alasan TCP header memiliki window field. Saat nanti mempelajari Congestion Control, batas pengiriman aktual menjadi minimum antara receiver-side limit dan network-side limit.

5.2.7 Triggering Transmission dan Kinerja TCP

Inti Konsep.

Triggering transmission menjawab kapan byte dalam buffer TCP harus dijadikan segmen. Kinerja TCP dipengaruhi oleh MSS, MTU, delay-bandwidth product, ukuran window, dan apakah algoritma menghindari segmen kecil yang boros overhead.

TCP tidak selalu mengirim setiap byte begitu aplikasi menulisnya. Byte stream dikumpulkan di buffer lalu dikirim ketika salah satu kondisi terjadi: data mencapai ****Maximum Segment Size (MSS)****, aplikasi meminta push, atau timer memaksa pengiriman.

MSS diturunkan dari MTU:

$$\text{MSS} = \text{MTU} - (\text{TCP header} + \text{IP header})$$

Tujuannya agar segmen TCP tidak menyebabkan fragmentasi IP pada link lokal.

Untuk menjaga link tetap sibuk, jumlah data in flight ideal mengikuti **delay-bandwidth product**:

$$\text{Data in flight ideal} \approx \text{Bandwidth} \times \text{RTT}$$

Jika window terlalu kecil dibanding nilai ini, TCP tidak mampu mengisi pipa jaringan. Sumber memberi contoh bahwa pada RTT 100 ms, T1 1.5 Mbps membutuhkan window sekitar 18 KB, Ethernet 10 Mbps sekitar 122 KB, dan OC-48 2.5 Gbps sekitar 29.6 MB.

Nagle's Algorithm mengurangi tinygrams:

1. Jika data tersedia dan ukuran window cukup untuk segmen MSS, kirim segmen penuh.
2. Jika data kecil dan masih ada data belum di-ACK, tahan data baru sampai ACK tiba.
3. Jika tidak ada data in flight, kirim data kecil tersebut.

Ekstensi TCP untuk jaringan cepat meliputi **window scaling**, **timestamp**, dan **PAWS**. Window scaling memperbesar window efektif melewati batas 16-bit lama. Timestamp membantu pengukuran RTT tanpa ambiguity. PAWS memakai timestamp untuk membedakan segmen lama dari segmen baru ketika sequence number 32-bit berpotensi wraparound.

Topik ini menghubungkan Transport Layer dengan soal performa: waktu transfer file dapat dihitung dari handshake, pertumbuhan slow start per RTT, batas receiver window, dan sisa data yang dikirim pada window maksimum.

5.2.8 RPC

Inti Konsep.

Remote Procedure Call (RPC) membuat komunikasi jaringan tampak seperti pemanggilan fungsi lokal. RPC penting karena menyembunyikan detail request/reply, pengemasan argumen, dan perbedaan representasi data antar-mesin dari programmer aplikasi.

Motivasi RPC adalah penyederhanaan aplikasi terdistribusi. Tanpa RPC, programmer harus menulis format pesan, mengirim request, menunggu reply, memeriksa error, dan menerjemahkan tipe data secara manual. RPC membungkus itu menjadi model pemanggilan prosedur.

Mekanismenya:

1. Client memanggil fungsi seolah-olah fungsi itu lokal.
2. **Client stub** menerima argumen dan melakukan **marshalling**, yaitu mengubah struktur data menjadi deretan byte jaringan.
3. Request dikirim ke server.
4. **Server stub** melakukan unmarshalling dan memanggil prosedur server yang sesungguhnya.
5. Hasil dikemas kembali, dikirim sebagai reply, lalu dibuka oleh client stub.

Format representasi data yang disebut sumber adalah **XDR (External Data Representation)** pada SunRPC dan **NDR (Network Data Representation)** pada DCE-RPC. SunRPC cenderung minimalis, sedangkan DCE-RPC lebih kompleks dan dapat menangani fragmentasi, ACK, dan kontrol yang lebih rinci.

RPC berhubungan dengan Application Layer karena banyak web services modern memakai gaya request/reply serupa, meskipun sering dibungkus di atas HTTP, REST, SOAP, atau format data lain.

5.2.9 RTP dan RTCP

Inti Konsep.

RTP menyediakan dukungan transport untuk multimedia real-time di atas UDP. RTP penting karena audio/video lebih membutuhkan timing, urutan media, dan sinkronisasi playback daripada retransmission penuh seperti TCP.

TCP tidak cocok untuk banyak aliran real-time karena retransmission dapat menyebabkan head-of-line blocking dan delay yang lebih merusak pengalaman daripada sedikit packet loss. RTP mengambil pendekatan berbeda: ia tidak menjamin recovery, tetapi memberi informasi agar aplikasi bisa memutar media dengan timing yang benar.

****Application Level Framing (ALF)**** adalah prinsip bahwa aplikasi paling memahami batas unit datanya. Karena itu, RTP memberi field seperti sequence number dan timestamp, tetapi membiarkan aplikasi menentukan cara menangani loss, jitter, codec, dan sinkronisasi.

Mekanisme RTP:

1. Aplikasi multimedia menghasilkan frame atau potongan audio/video.
2. RTP menambahkan sequence number untuk mendeteksi loss dan reorder.
3. RTP menambahkan timestamp untuk sinkronisasi waktu playback.
4. Paket RTP dikirim di atas UDP agar delay rendah.
5. RTCP mengirim feedback seperti loss rate, delay, dan informasi sinkronisasi.

RTP berkaitan dengan UDP karena memakai layanan datagram yang cepat, dan berkaitan dengan Congestion Control/QoS karena multimedia real-time tetap harus memperhatikan dampak lalu lintasnya pada jaringan.

5.2.10 SCTP

Inti Konsep.

SCTP adalah alternatif desain protokol transport yang disebut sumber sebagai ruang desain lain selain TCP dan UDP. Konsep ini penting untuk menunjukkan bahwa layanan transport tidak hanya dua ekstrem: datagram sederhana atau byte stream TCP.

Sumber menyebut SCTP sebagai protokol alternatif, tetapi tidak memberi penjelasan sedalam UDP dan TCP. Secara konseptual, SCTP memperlihatkan bahwa kebutuhan aplikasi dapat mendorong desain transport baru, misalnya ketika aplikasi membutuhkan fitur yang tidak pas dengan byte stream TCP atau datagram UDP.

Komentar 5.1: Keterbatasan Sumber

Informasi SCTP di sumber utama terbatas pada penyebutan sebagai alternatif desain. Penjelasan detail seperti multi-streaming, multi-homing, association, dan chunk tidak dikembangkan di sumber NotebookLM yang digunakan untuk materi ini, sehingga catatan ini tidak menjadikannya fokus utama ujian.

SCTP paling berguna dipahami sebagai konteks: TCP dan UDP adalah protokol dominan, tetapi bukan satu-satunya kemungkinan rancangan transport.

5.2.11 QUIC sebagai Transport Modern Berbasis UDP[†]

Inti Konsep.

QUIC adalah tambahan pengetahuan umum tentang transport modern yang berjalan di atas UDP, tetapi membangun stream, flow control, security, loss recovery, dan congestion control sendiri. QUIC penting sebagai contoh bahwa aplikasi modern dapat memperoleh fitur mirip

TCP sekaligus menghindari beberapa keterbatasan TCP klasik.

Komentar 5.2: Catatan: Sumber Pengetahuan Umum

Informasi QUIC tidak ditemukan sebagai topik utama pada hasil NotebookLM Transport Layer dan ditambahkan dari RFC 9000 serta RFC 9114. QUIC menyediakan stream yang flow-controlled, connection establishment rendah latensi, migration antar-network path, serta integrasi TLS untuk confidentiality dan integrity. HTTP/3 memetakan semantik HTTP di atas QUIC sehingga stream HTTP dapat berjalan secara independen; packet loss pada satu stream tidak harus memblokir stream lain seperti pada multiplexing di atas satu byte stream TCP.

Dalam peta konsep, QUIC mengikat Transport Layer, Application Layer, dan Network Security. Ia memakai UDP agar dapat berjalan di infrastruktur Internet yang sudah mendukung UDP, tetapi menerapkan banyak fungsi transport di atasnya.

5.2.12 Relasi Lintas Materi

Inti Konsep.

Transport Layer adalah titik temu aplikasi, reliability, congestion, wireless loss, dan security. Memahami relasi ini penting karena soal jaringan sering meminta alasan pemilihan TCP/UDP, dampak loss pada TCP, atau posisi TLS terhadap aplikasi dan transport.

Dengan **Application Layer**, transport menentukan sifat saluran komunikasi. HTTP, SMTP, FTP, Telnet, dan banyak web services memakai TCP karena data harus utuh dan berurutan. DNS, SNMP, RTP, dan VoIP sering memakai UDP karena transaksi pendek atau timing lebih penting daripada retransmission penuh.

Dengan **Congestion Control**, TCP memandang loss, timeout, dan duplicate ACK sebagai sinyal penting. Pada materi Congestion Control, sinyal ini akan dipakai untuk mengatur CongestionWindow melalui slow start, AIMD, fast retransmit, dan fast recovery.

Dengan **Wireless Network**, TCP menghadapi masalah interpretasi. Packet loss pada jaringan kabel sering diakibatkan congestion, tetapi pada wireless loss bisa terjadi karena path loss, interference, atau bit error. Jika TCP salah menganggap wireless loss sebagai congestion, TCP menurunkan window dan throughput turun tanpa alasan congestion yang benar.

Dengan **Network Security**, TCP dan UDP tidak otomatis memberi confidentiality atau authentication. TLS/SSL diletakkan di atas transport untuk mengamankan aplikasi seperti HTTPS. Pada QUIC, integrasi security menjadi bagian dari desain transport modern.

5.3 Algoritma dan Rumus Penting

5.3.1 UDP Checksum

Makna Rumus.

UDP checksum digunakan untuk mendeteksi error bit dan misdelivery endpoint. Checksum tidak memperbaiki error, tetapi memberi dasar bagi penerima untuk mengetahui bahwa datagram tidak valid.

Listing 1: Pseudocode UDP checksum

```
1 Input: UDP header, UDP data, IP pseudoheader
2 Output: checksum field
3
4 sender:
5     words = split_into_16_bit_words(pseudoheader + udp_header + data)
6     sum = ones_complement_sum(words)
7     checksum = ones_complement(sum)
8     write checksum into UDP header
```

```

9
10 receiver:
11     words = split_into_16_bit_words(pseudoheader + udp_header + data)
12     sum = ones_complement_sum(words)
13     if sum == all_ones:
14         accept as no detected error
15     else:
16         report detected error

```

5.3.2 TCP Four-Tuple Demultiplexing

Makna Rumus.

Four-tuple TCP membedakan koneksi yang berbeda walaupun beberapa koneksi menuju server dan port tujuan yang sama. Rumus ini dipakai sistem operasi saat menyerahkan segmen TCP ke socket yang benar.

$$\text{TCPConnection} = \langle \text{SrcPort}, \text{SrcIPAddr}, \text{DstPort}, \text{DstIPAddr} \rangle \quad (1)$$

dengan:

- SrcPort: port TCP pengirim.
- SrcIPAddr: alamat IP pengirim.
- DstPort: port TCP tujuan.
- DstIPAddr: alamat IP tujuan.

5.3.3 Adaptive RTT Estimation

Makna Rumus.

Estimasi RTT menentukan kapan TCP harus menganggap segmen hilang. Rumus ini digunakan agar timeout mengikuti kondisi jaringan aktual.

$$\text{EstimatedRTT} = \alpha \cdot \text{EstimatedRTT} + (1 - \alpha) \cdot \text{SampleRTT} \quad (2)$$

$$\text{TimeOut} = 2 \cdot \text{EstimatedRTT} \quad (3)$$

dengan:

- SampleRTT: RTT dari satu segmen yang diukur.
- EstimatedRTT: rata-rata bergerak RTT.
- α : faktor penghalus, lazim sekitar 0.8 sampai 0.9 pada sumber.

Versi Jacobson/Karels:

$$\text{Difference} = \text{SampleRTT} - \text{EstimatedRTT} \quad (4)$$

$$\text{EstimatedRTT} = \text{EstimatedRTT} + \delta \cdot \text{Difference} \quad (5)$$

$$\text{Deviation} = \text{Deviation} + \delta \cdot (|\text{Difference}| - \text{Deviation}) \quad (6)$$

$$\text{TimeOut} = \mu \cdot \text{EstimatedRTT} + \phi \cdot \text{Deviation} \quad (7)$$

5.3.4 Flow Control Window

Makna Rumus.

Flow-control window memastikan sender tidak mengirim lebih banyak byte daripada ruang receiver yang tersedia. Rumus ini dipakai untuk menentukan apakah TCP boleh mengirim data baru.

$$\text{RcvWindow} = \text{MaxRcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead}) \quad (8)$$

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{AdvertisedWindow} \quad (9)$$

$$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAcked}) \quad (10)$$

5.3.5 MSS dan Delay-Bandwidth Product

Makna Rumus.

MSS menentukan ukuran payload TCP terbesar yang aman terhadap MTU lokal, sedangkan delay-bandwidth product menentukan jumlah data ideal yang harus berada in flight agar link tidak menganggur.

$$\text{MSS} = \text{MTU} - (\text{TCPHeader} + \text{IPHeader}) \quad (11)$$

$$\text{WindowIdeal} \approx \text{Bandwidth} \times \text{RTT} \quad (12)$$

$$\text{WraparoundTime} = \frac{2^{32}}{\text{Bandwidth}} \quad (13)$$

dengan Bandwidth pada Persamaan 13 harus dinyatakan dalam byte per detik agar satuannya menjadi detik.

5.3.6 Nagle's Algorithm

Tujuan Algoritma.

Nagle's Algorithm mengurangi tinygrams, yaitu segmen kecil yang membuang bandwidth karena overhead header terlalu besar dibanding payload. Algoritma ini menahan data kecil jika masih ada data belum di-ACK.

Listing 2: Pseudocode Nagle's Algorithm

```
1 Input: new data from application, MSS, bytes_in_flight
2 Output: send now or buffer
3
4 if buffered_data_size >= MSS:
5     send_full_segment()
6 else if bytes_in_flight > 0:
7     buffer_until_ack_arrives()
8 else:
9     send_current_data()
```

6 Congestion Control

Congestion Control membahas bagaimana jaringan mengatur laju pengiriman agar router, switch, dan link tidak kewalahan oleh paket yang terlalu banyak. Materi ini berdiri di atas Transport Layer: TCP menggunakan ACK, timeout, duplicate ACK, dan window untuk memperkirakan kondisi jaringan, lalu menyesuaikan laju pengiriman.

Fokus belajar utama adalah membedakan flow control dan congestion control, memahami fairness serta resource allocation, menguasai algoritma TCP seperti AIMD, Slow Start, Fast Retransmit, dan Fast Recovery, lalu melihat bagaimana router dan QoS membantu aplikasi real-time yang tidak cukup dilayani oleh best-effort Internet.

6.1 Outline Fundamental Konsep Materi

1. **WAJIB** Konsep Dasar Alokasi Sumber Daya dan Congestion
 - (a) **WAJIB** Bandwidth, buffer, packet contention, queue overflow, dan packet drop
 - (b) **WAJIB** Bottleneck router dan congestion collapse
 - (c) **WAJIB** Flow dan model packet-switched connectionless network
 - (d) **PAHAM** Taksonomi router-centric/host-centric, reservation-based/feedback-based, window-based/rate-based
2. **WAJIB** Kriteria Evaluasi
 - (a) **WAJIB** Throughput, delay, dan power
 - (b) **WAJIB** Jain's Fairness Index
 - (c) **PAHAM** Max-min fairness[†]
3. **WAJIB** Disiplin Antrean Router
 - (a) **WAJIB** Scheduling discipline dan drop policy
 - (b) **WAJIB** FIFO dan tail drop
 - (c) **WAJIB** Fair Queuing dan timestamp finish time
 - (d) **PAHAM** Weighted Fair Queuing
4. **WAJIB** TCP Congestion Control
 - (a) **WAJIB** CongestionWindow, AdvertisedWindow, MaxWindow, dan EffectiveWindow
 - (b) **WAJIB** Self-clocking dan loss sebagai sinyal congestion
 - (c) **WAJIB** AIMD: additive increase dan multiplicative decrease
 - (d) **WAJIB** Slow Start dan ssthresh/CongestionThreshold
 - (e) **WAJIB** Fast Retransmit dan Fast Recovery
 - (f) **PAHAM** TCP Tahoe dan TCP Reno
5. **WAJIB** Congestion Avoidance Lanjutan
 - (a) **PAHAM** DECbit
 - (b) **WAJIB** RED: AvgLen, threshold, dan drop probability
 - (c) **PAHAM** TCP Vegas
 - (d) **PAHAM** ECN[†]
6. **WAJIB** Quality of Service
 - (a) **WAJIB** Application requirements: elastic, real-time, tolerant, intolerant, adaptive
 - (b) **WAJIB** IntServ/RSVP: Flowspec, TSpec, RSpec, token bucket, admission control, PATH/RESV, soft state

- (c) **WAJIB** DiffServ: PHB, EF, AF, Weighted RED/RIO
 - (d) **PAHAM** Equation-based congestion control
 - (e) **PAHAM** Leaky bucket[†]
7. **WAJIB** Relasi Lintas Materi
- (a) **WAJIB** Flow control vs congestion control
 - (b) **WAJIB** Transport Layer dan TCP
 - (c) **PAHAM** Application Layer, UDP multimedia, QoS
 - (d) **WAJIB** Wireless loss vs congestion loss

6.2 Penjelasan Konsep

6.2.1 Konsep Dasar Alokasi Sumber Daya dan Congestion

Inti Konsep.

Congestion terjadi ketika terlalu banyak paket bersaing memakai sumber daya jaringan yang terbatas, terutama bandwidth link dan buffer router. Konsep ini penting karena tanpa kontrol, packet drop memicu retransmission yang justru dapat membuat jaringan mengalami congestion collapse.

Motivasi utama congestion control adalah sifat jaringan packet-switched yang digunakan bersama oleh banyak flow. Setiap host mungkin merasa link lokalnya masih mampu mengirim cepat, tetapi paket dari banyak host dapat bertemu di satu router yang sama dan berebut link keluaran yang lebih lambat. Titik semacam ini disebut **“bottleneck router”**, yaitu router atau link yang menjadi pembatas kapasitas jalur.

Secara formal, **“resource allocation”** adalah proses pembagian sumber daya jaringan kepada permintaan aplikasi yang saling bersaing. Sumber daya utama adalah **“bandwidth”**, yaitu laju transmisi link, dan **“buffer”**, yaitu memori antrean pada router/switch. **“Congestion”** adalah keadaan ketika laju kedatangan paket ke suatu router melebihi laju layanan keluarannya sehingga antrean tumbuh, buffer penuh, lalu paket harus dibuang.

Mekanisme dasar terjadinya congestion:

1. Banyak host mengirim paket ke jaringan packet-switched.
2. Paket dari beberapa flow bertemu di router yang memiliki link keluaran sama.
3. Jika laju kedatangan lebih tinggi daripada laju transmisi, router memasukkan paket ke buffer.
4. Jika buffer penuh, terjadi **“queue overflow”**.
5. Router melakukan **“packet drop”**.
6. Host TCP mendeteksi loss melalui timeout atau duplicate ACK, lalu menurunkan laju; jika tidak ada kontrol, retransmission dapat memperparah congestion.

Istilah	Definisi atau Peran
Bandwidth	Kapasitas laju data pada link jaringan.
Buffer	Memori router/switch untuk menampung paket yang menunggu dikirim.
Flow	Rangkaian paket antara pasangan sumber-tujuan, dapat dipandang host-to-host atau process-to-process.
Packet contention	Persaingan banyak paket untuk link keluaran yang sama.
Queue overflow	Kondisi saat buffer router penuh.
Bottleneck router	Router atau link yang menjadi titik penyempitan utama jalur.
Congestion collapse	Kondisi ketika throughput efektif jatuh drastis karena loss dan retransmission saling memperparah.

Tabel 7: Istilah dasar congestion dan resource allocation

Pendekatan congestion control dapat diklasifikasikan menjadi beberapa dimensi. **Router-centric** berarti router aktif menjadwalkan, membuang, atau menandai paket. **Host-centric** berarti end host menyesuaikan laju dari sinyal yang diamati. **Reservation-based** meminta kapasitas sebelum mengirim, sedangkan **feedback-based** mengirim dulu lalu menyesuaikan dari sinyal eksplisit atau implisit. **Window-based** membatasi data in flight, sedangkan **rate-based** membatasi laju bit per detik.

Konsep ini menjadi dasar semua subtopik berikut. Queuing discipline mengatur router saat congestion mulai muncul; TCP congestion control mengatur host; QoS mencoba memberi jaminan layanan untuk flow tertentu.

6.2.2 Kriteria Evaluasi

Inti Konsep.

Kriteria evaluasi congestion control mengukur apakah alokasi sumber daya efektif dan adil. Efektivitas menyeimbangkan throughput tinggi dan delay rendah, sedangkan fairness memastikan flow yang bersaing tidak saling memonopoli bandwidth.

Congestion control tidak cukup hanya membuat jaringan tidak macet. Jika algoritma terlalu konservatif, throughput rendah. Jika terlalu agresif, delay antrean naik dan packet drop meningkat. Karena itu, sumber memakai dua sudut pandang: **effective resource allocation** dan **fair resource allocation**.

Metrik efektivitas yang disebut sumber adalah **power**, yaitu keseimbangan antara throughput dan delay. Bentuk umum:

$$\text{Power} = \frac{\text{Throughput}^\alpha}{\text{Delay}}, \quad 0 < \alpha \leq 1$$

Jika $\alpha = 1$, bentuknya menjadi throughput dibagi delay. Throughput terlalu tinggi dengan delay sangat besar tidak ideal; delay rendah dengan throughput hampir nol juga tidak ideal.

Fairness diukur dengan **Jain's Fairness Index**:

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$$

dengan:

- x_i : throughput yang diterima flow ke- i .
- n : jumlah flow.

Nilainya berada antara 0 dan 1. Nilai 1 berarti alokasi sangat adil, yaitu semua flow mendapat throughput yang sama.

Komentar 6.1: Catatan: Sumber Pengetahuan Umum

Max-min fairness ditambahkan dari silabus/topik standar congestion control. Prinsipnya: alokasi dianggap max-min fair jika tidak ada flow yang dapat dinaikkan lajunya tanpa menurunkan flow lain yang sudah memiliki laju sama atau lebih kecil. Konsep ini berguna untuk memahami fairness saat flow memiliki bottleneck berbeda dan tidak selalu dapat dibagi rata secara global.

Kriteria evaluasi ini berkaitan langsung dengan Fair Queuing, AIMD, dan QoS. Fair Queuing mencoba menegakkan fairness di router, AIMD mencoba membuat TCP stabil dan adil antar-koneksi, sedangkan QoS sengaja memberi alokasi berbeda ketika aplikasi memiliki kebutuhan layanan berbeda.

6.2.3 Disiplin Antrean Router

Inti Konsep.

Disiplin antrean adalah mekanisme router untuk menentukan paket mana yang dikirim berikutnya dan paket mana yang dibuang saat buffer penuh. Konsep ini penting karena router adalah titik tempat contention benar-benar terjadi.

Motivasi queuing discipline adalah buffer router terbatas. Jika router hanya memakai antrean sederhana, flow agresif dapat memonopoli buffer dan bandwidth. Router perlu **scheduling discipline** untuk memilih urutan transmisi dan **drop policy** untuk memilih paket yang dibuang ketika buffer penuh.

Jenis utama yang perlu dipahami:

Disiplin	Definisi atau Peran
FIFO	First-In First-Out; paket pertama datang dikirim pertama.
Tail drop	Paket baru dibuang jika tiba saat buffer sudah penuh.
Priority Queuing	Paket dibagi ke kelas prioritas; antrean prioritas tinggi dilayani lebih dahulu.
Fair Queuing	Router membuat antrean per-flow agar flow agresif tidak mendominasi flow lain.
Weighted Fair Queuing	Fair Queuing dengan bobot sehingga flow/kelas tertentu mendapat proporsi bandwidth lebih besar.

Tabel 8: Jenis disiplin antrean router

Fair Queuing idealnya melakukan **bit-by-bit round-robin**, yaitu setiap flow dilayani satu bit bergiliran. Karena router nyata tidak dapat menginterupsi paket di tengah transmisi, FQ mensimulasikan waktu selesai teoritis setiap paket:

$$F_i = \max(F_{i-1}, A_i) + P_i$$

dengan:

- F_i : finish time teoritis paket ke- i pada flow tersebut.
- F_{i-1} : finish time paket sebelumnya pada flow yang sama.
- A_i : arrival time paket ke- i .
- P_i : panjang paket ke- i .

Router lalu memilih paket dengan F_i terkecil untuk dikirim berikutnya.

Weighted Fair Queuing memberi bobot pada flow. Jika suatu kelas diberi bobot lebih besar, ia dapat memperoleh proporsi bandwidth yang lebih besar. Ini menjadi dasar teknis untuk QoS, khususnya layanan premium seperti Expedited Forwarding.

Queuing discipline menghubungkan router-centric control dengan TCP. TCP end-host tetap penting, tetapi router dapat mencegah flow UDP atau flow tidak patuh mengambil seluruh kapasitas.

6.2.4 TCP Congestion Control

Inti Konsep.

TCP Congestion Control membatasi jumlah data yang boleh berada di jaringan dengan CongestionWindow. Algoritma seperti AIMD, Slow Start, Fast Retransmit, dan Fast Recovery membuat TCP dapat memanfaatkan bandwidth tanpa terus-menerus menyebabkan congestion collapse.

Sebelum mekanisme ini diperkenalkan, host TCP mengirim secepat AdvertisedWindow penerima mengizinkan. Jika router bottleneck membuang paket, host melakukan retransmission, lalu beban jaringan makin berat. TCP Congestion Control menambahkan batas dari sisi jaringan: **CongestionWindow (cwnd)**.

Batas aktual pengiriman TCP:

$$\text{MaxWindow} = \min(\text{CongestionWindow}, \text{AdvertisedWindow})$$

$$\text{EffectiveWindow} = \text{MaxWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

AdvertisedWindow melindungi receiver, sedangkan CongestionWindow melindungi jaringan.

Prinsip pentingnya adalah **self-clocking**. ACK yang kembali ke sender dianggap sebagai tanda bahwa sebagian data sudah keluar dari jaringan, sehingga sender boleh memasukkan data baru. TCP juga menganggap packet loss sebagai sinyal congestion karena pada jaringan kabel, loss lebih sering berasal dari queue overflow daripada bit error.

Mekanisme utama:

1. **Multiplicative Decrease**: saat loss/timeout, cwnd dipotong, umumnya menjadi setengah, tetapi tidak di bawah 1 MSS.
2. **Additive Increase**: saat ACK terus sukses, cwnd dinaikkan perlahan, kira-kira 1 MSS per RTT.
3. **Slow Start**: koneksi baru mulai dari cwnd kecil lalu tumbuh eksponensial, menggandakan cwnd tiap RTT sampai mencapai ssthresh.
4. **Fast Retransmit**: tiga duplicate ACK menjadi sinyal kuat paket hilang, sehingga sender retransmit tanpa menunggu timeout.
5. **Fast Recovery**: setelah Fast Retransmit, cwnd tidak dijatuhkan ke 1 MSS, tetapi dipotong sebagian dan lanjut tumbuh linier.

Kenaikan additive per ACK:

$$\text{Increment} = \text{MSS} \times \frac{\text{MSS}}{\text{CongestionWindow}}$$

$$\text{CongestionWindow} \leftarrow \text{CongestionWindow} + \text{Increment}$$

Pseudocode kenaikan cwnd dari sumber:

Listing 3: Pseudocode kenaikan CongestionWindow

```
1 cw = state.CongestionWindow
2 incr = state.maxseg          # MSS
3
4 if cw > state.CongestionThreshold:
5     incr = incr * incr / cw
6
7 state.CongestionWindow = min(cw + incr, TCP_MAXWIN)
```

Varian historis:

Varian	Perilaku Penting
TCP Tahoe	Menggunakan Slow Start, AIMD, Fast Retransmit; saat loss, cwnd kembali ke 1 MSS.
TCP Reno	Menambahkan Fast Recovery sehingga setelah tiga duplicate ACK cwnd tidak jatuh penuh ke 1 MSS.

Tabel 9: Perbandingan TCP Tahoe dan TCP Reno

TCP congestion control berkaitan langsung dengan fairness. AIMD menghasilkan pola kenaikan linier dan penurunan multiplikatif yang secara teoritis mendorong flow TCP yang berbagi bottleneck menuju pembagian kapasitas yang stabil.

6.2.5 Congestion Avoidance Lanjutan

Inti Konsep.

Congestion avoidance mencoba memberi sinyal sebelum router benar-benar penuh dan mulai membuang banyak paket. Konsep ini penting karena TCP klasik sering bereaksi setelah loss terjadi, padahal loss berarti jaringan sudah melewati titik sehatnya.

Motivasi congestion avoidance adalah TCP Reno/Tahoe bersifat reaktif: cwnd dinaikkan sampai jaringan memberi sinyal loss. Alternatif lanjutan mencoba mendeteksi antrean yang mulai tumbuh dan memberi sinyal dini.

****DECbit**** adalah pendekatan eksplisit. Router menghitung rata-rata panjang antrean. Jika rata-rata antrean melewati ambang, router menandai congestion bit pada header paket. Host melihat persentase paket bertanda dalam satu window: jika kurang dari 50 persen, window dinaikkan; jika 50 persen atau lebih, window dikalikan 0.875.

****Random Early Detection (RED)**** adalah pendekatan router-centric yang sengaja membuang sebagian paket sebelum buffer penuh. Router menghitung rata-rata antrean:

$$\text{AvgLen} = (1 - \text{Weight}) \times \text{AvgLen} + \text{Weight} \times \text{SampleLen}$$

Jika $\text{AvgLen} \leq \text{MinThreshold}$, paket diterima. Jika $\text{AvgLen} \geq \text{MaxThreshold}$, paket dibuang. Jika berada di antara keduanya, paket dibuang dengan probabilitas:

$$\text{TempP} = \text{MaxP} \times \frac{\text{AvgLen} - \text{MinThreshold}}{\text{MaxThreshold} - \text{MinThreshold}}$$

$$P = \frac{\text{TempP}}{1 - \text{count} \times \text{TempP}}$$

dengan count adalah jumlah paket yang lolos sejak drop acak terakhir.

****TCP Vegas**** adalah pendekatan source-based. Host mengamati RTT, membandingkan laju yang diharapkan dengan laju aktual:

$$\text{ExpectedRate} = \frac{\text{CongestionWindow}}{\text{BaseRTT}}$$

$$\text{ActualRate} = \frac{\text{DataSent}}{\text{SampleRTT}}$$

$$\text{Diff} = \text{ExpectedRate} - \text{ActualRate}$$

Jika Diff kecil di bawah α , window dinaikkan. Jika Diff besar di atas β , window diturunkan. Intuisinya: kenaikan RTT menunjukkan antrean mulai terbentuk.

Komentar 6.2: Catatan: Sumber Pengetahuan Umum

Explicit Congestion Notification (ECN) ditambahkan dari RFC 3168 sebagai mekanisme eksplisit modern. ECN memungkinkan router menandai paket sebagai mengalami congestion tanpa harus membuangnya, selama endpoint mendukung ECN. Ini memperhalus ide seperti DECbit: sinyal congestion dapat dikirim melalui marking, bukan hanya packet loss.

Congestion avoidance menghubungkan router dan host. RED dan ECN memberi sinyal dari jaringan; Vegas mencoba menyimpulkan sinyal dari RTT; TCP Reno/Tahoe bereaksi pada loss.

6.2.6 Quality of Service

Inti Konsep.

Quality of Service (QoS) adalah kemampuan jaringan memberi tingkat layanan berbeda untuk flow atau kelas trafik tertentu. QoS penting karena aplikasi real-time tidak cukup hanya diberi layanan best-effort yang adil rata; mereka membutuhkan batas delay, jitter, loss, atau bandwidth.

Motivasi QoS berasal dari kebutuhan aplikasi. Transfer file adalah **elastic application**: makin banyak bandwidth makin baik, tetapi delay tidak harus ketat. VoIP, video conference, dan kontrol real-time membutuhkan **timeliness**. Paket yang datang terlambat pada audio/video sering tidak berguna walaupun akhirnya tiba.

Klasifikasi kebutuhan aplikasi:

Kelas Aplikasi	Makna
Elastic	Dapat menyesuaikan diri dengan bandwidth yang tersedia, misalnya file transfer.
Real-time	Sensitif terhadap waktu kedatangan paket.
Tolerant	Masih berfungsi meskipun ada loss atau delay terbatas.
Intolerant	Membutuhkan batas layanan ketat; loss/delay besar tidak dapat diterima.
Adaptive	Dapat menyesuaikan rate atau delay playback sesuai kondisi jaringan.

Tabel 10: Kebutuhan aplikasi untuk QoS

Integrated Services (IntServ) memberi jaminan fine-grained per-flow. Aplikasi mengirim **Flowspec** yang terdiri dari **TSpec** dan **RSpec**. TSpec menjelaskan karakteristik trafik, sedangkan RSpec menjelaskan layanan yang diminta. Router menjalankan **admission control** untuk menentukan apakah flow baru dapat diterima tanpa merusak jaminan flow lama.

TSpec dapat dinyatakan dengan **token bucket**:

- r : token rate, jumlah token yang masuk per detik.
- B : bucket depth, jumlah token maksimum yang dapat ditabung.

Mengirim 1 byte menghabiskan 1 token. Dalam jangka panjang, laju rata-rata tidak melebihi r , tetapi burst sementara masih bisa dilakukan sampai batas B .

RSVP melakukan signaling reservasi. Pengirim mengirim PATH ke penerima; penerima mengirim RESV balik melalui router di jalur tersebut. State reservasi bersifat **soft state**, artinya akan hilang jika tidak diperbarui berkala.

Differentiated Services (DiffServ) memberi layanan coarse-grained berbasis kelas. Router tidak menyimpan state per-flow; paket diberi markah kelas, lalu router menerapkan **Per-Hop Behavior (PHB)**. **Expedited Forwarding (EF)** ditujukan untuk layanan delay rendah seperti voice. **Assured Forwarding (AF)** memberi probabilitas perlakuan lebih baik dan sering memakai Weighted RED/RIO untuk membedakan paket in-profile dan out-of-profile.

****Equation-based congestion control**** memodelkan laju pengiriman agar aplikasi UDP multimedia tetap TCP-friendly:

$$\text{Rate} \propto \frac{1}{\text{RTT}\sqrt{\rho}}$$

dengan ρ adalah probabilitas packet loss. Intuisinya: jika RTT atau loss naik, rate harus turun.

Komentar 6.3: Catatan: Sumber Pengetahuan Umum

Leaky bucket ditambahkan dari topik standar congestion control dan traffic shaping. Berbeda dari token bucket yang mengizinkan burst selama token tersedia, leaky bucket mengeluarkan paket pada laju relatif tetap sehingga trafik menjadi lebih halus. Konsep ini berguna untuk memahami rate-based shaping, tetapi pada sumber utama QoS lebih banyak ditekankan melalui token bucket.

QoS berkaitan dengan Application Layer karena pilihan layanan bergantung pada kebutuhan aplikasi. QoS juga berkaitan dengan UDP/RTP karena aplikasi real-time sering memilih UDP untuk menghindari delay retransmission TCP, tetapi tetap membutuhkan kontrol agar tidak merusak jaringan.

6.2.7 Relasi Lintas Materi

Inti Konsep.

Congestion Control menghubungkan Transport Layer, router, aplikasi real-time, wireless, dan security. Relasi ini penting karena packet loss tidak selalu berarti hal yang sama di semua media, dan tidak semua aplikasi dapat dilayani dengan strategi TCP best-effort.

Dengan ****Transport Layer****, congestion control adalah pelengkap flow control. Flow control membatasi sender berdasarkan kapasitas receiver melalui AdvertisedWindow. Congestion control membatasi sender berdasarkan kapasitas jaringan melalui CongestionWindow.

Dengan ****Application Layer****, congestion control menentukan apakah aplikasi harus memakai TCP, UDP dengan RTP, atau layanan QoS. Aplikasi real-time sering menghindari retransmission TCP, tetapi tetap memerlukan rate control atau QoS agar trafiknya tidak merusak jaringan.

Dengan ****Wireless Network****, loss dapat berasal dari bit error, path loss, atau interference. TCP yang menganggap loss sebagai congestion dapat salah menurunkan cwnd, sehingga throughput turun walaupun router tidak macet. ECN atau sinyal eksplisit dapat membantu membedakan congestion router dari loss non-congestion.

Dengan ****Network Security****, serangan availability seperti DDoS dan worm dapat membanjiri bandwidth serta buffer, menimbulkan congestion buatan. Karena itu, congestion control tidak menggantikan firewall atau filtering, tetapi dampaknya terasa langsung pada throughput dan delay jaringan.

6.3 Algoritma dan Rumus Penting

6.3.1 Jain's Fairness Index

Makna Rumus.

Jain's Fairness Index mengukur seberapa merata throughput antar-flow. Rumus ini digunakan untuk mengevaluasi apakah sebuah mekanisme congestion control atau queuing memberi pembagian bandwidth yang adil.

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2} \quad (14)$$

dengan:

- x_i : throughput flow ke- i .

- n : jumlah flow yang dibandingkan.

6.3.2 Fair Queuing Finish Time

Makna Rumus.

Finish time Fair Queuing mensimulasikan bit-by-bit round-robin tanpa harus memotong paket menjadi bit fisik yang diselingi. Router mengirim paket dengan finish time teoritis terkecil.

$$F_i = \max(F_{i-1}, A_i) + P_i \quad (15)$$

dengan:

- F_i : finish time paket ke- i .
- F_{i-1} : finish time paket sebelumnya pada flow yang sama.
- A_i : arrival time paket ke- i .
- P_i : panjang paket ke- i .

6.3.3 AIMD dan Slow Start

Tujuan Algoritma.

AIMD menjaga TCP tetap stabil dan adil, sedangkan Slow Start mempercepat pencarian kapasitas awal. Keduanya mengatur CongestionWindow berdasarkan ACK dan loss.

Listing 4: Pseudocode TCP congestion control ringkas

```

1 on new_connection:
2     cwnd = 1 * MSS
3     ssthresh = large_value
4
5 on ACK:
6     if cwnd < ssthresh:
7         # Slow Start: exponential growth
8         cwnd = cwnd + MSS
9     else:
10        # Congestion Avoidance: additive increase
11        cwnd = cwnd + MSS * (MSS / cwnd)
12
13 on timeout:
14     ssthresh = cwnd / 2
15     cwnd = 1 * MSS
16
17 on three_duplicate_ACKs:
18     retransmit_missing_segment()
19     ssthresh = cwnd / 2
20     cwnd = ssthresh      # Reno-style fast recovery intuition

```

6.3.4 RED Average Queue Length dan Drop Probability

Makna Rumus.

RED memakai rata-rata antrean untuk memberi sinyal dini sebelum buffer penuh. Probabilitas drop naik saat AvgLen bergerak dari MinThreshold menuju MaxThreshold.

$$\text{AvgLen} = (1 - \text{Weight})\text{AvgLen} + \text{WeightSampleLen} \quad (16)$$

$$\text{TempP} = \text{MaxP} \frac{\text{AvgLen} - \text{MinThreshold}}{\text{MaxThreshold} - \text{MinThreshold}} \quad (17)$$

$$P = \frac{\text{TempP}}{1 - \text{countTempP}} \quad (18)$$

6.3.5 Token Bucket

Makna Rumus.

Token bucket membatasi laju rata-rata trafik tetapi tetap mengizinkan burst sementara. Rumus ini digunakan dalam TSpec pada IntServ untuk menyatakan karakteristik trafik flow.

$$\text{BytesSent}(T) \leq rT + B \quad (19)$$

dengan:

- r : token rate atau laju rata-rata jangka panjang.
- B : bucket depth atau burst maksimum.
- T : interval waktu observasi.

7 Application Layer

Application Layer adalah tempat protokol jaringan terlihat langsung sebagai aplikasi: web, email, DNS, file transfer, web services, P2P, dan multimedia. Materi ini menekankan bagaimana proses di end host saling bertukar pesan dengan aturan sintaks, semantik, tipe pesan, dan urutan respons yang disepakati.

Fokus ujian adalah memahami arsitektur aplikasi, DNS sebagai sistem direktori Internet, HTTP sebagai protokol web, email sebagai sistem asinkron push/pull, serta aplikasi lanjutan seperti CDN, REST/SOAP, BitTorrent, SNMP, SIP, dan RTP. Semua konsep ini bergantung pada pemilihan layanan Transport Layer yang tepat.

7.1 Outline Fundamental Konsep Materi

1. **WAJIB** Fondasi Application Layer
 - (a) **WAJIB** Proses aplikasi terdistribusi di end hosts dan user space
 - (b) **WAJIB** Client-server, peer-to-peer, dan hybrid architecture
 - (c) **WAJIB** Socket API, IP address, port number, dan process addressing
 - (d) **WAJIB** Kebutuhan layanan: data loss, timing, dan bandwidth
2. **WAJIB** Domain Name System (DNS)
 - (a) **WAJIB** Hostname-to-IP translation, host aliasing, mail aliasing, dan load distribution
 - (b) **WAJIB** Root, TLD, authoritative name server, dan local resolver
 - (c) **WAJIB** Iterative query, recursive query, caching, dan TTL
 - (d) **WAJIB** Resource Record A/AAAA, CNAME, NS, MX, PTR, dan TXT
 - (e) **PAHAM** nslookup, dig, dan non-authoritative answer
 - (f) **PAHAM** DNSSEC[†]
3. **WAJIB** Web dan HTTP
 - (a) **WAJIB** Web object, base HTML, dan URL
 - (b) **WAJIB** HTTP sebagai pull protocol di atas TCP dan sifat stateless
 - (c) **WAJIB** Non-persistent HTTP dan rumus waktu respons
 - (d) **WAJIB** Persistent HTTP, keep-alive, dan pipelining
 - (e) **WAJIB** Request/response format, method, status code, header, dan body
 - (f) **WAJIB** Cookies, web cache/proxy, Conditional GET, dan ETag
 - (g) **PAHAM** HTTP/2 dan HTTP/3[†]
4. **WAJIB** Electronic Mail
 - (a) **WAJIB** User agent, mail server, mailbox, dan message queue
 - (b) **WAJIB** SMTP sebagai push protocol di atas TCP port 25
 - (c) **WAJIB** RFC 822, MIME, Base64, dan multipart
 - (d) **WAJIB** POP3, IMAP, dan Webmail
 - (e) **WAJIB** Ketergantungan email pada DNS MX record
5. **PAHAM** Arsitektur Lanjutan
 - (a) **PAHAM** Overlay networks
 - (b) **WAJIB** CDN, surrogate server, dan DNS redirection
 - (c) **PAHAM** Web Services: REST dan SOAP
 - (d) **WAJIB** P2P: Gnutella, BitTorrent, dan DHT

6. **PAHAM** Protokol Aplikasi Khusus
 - (a) **PAHAM** FTP
 - (b) **PAHAM** SNMP, MIB, GET, GETNEXT, SET, dan TRAP
 - (c) **PAHAM** SDP, SIP, H.323, RTP, dan RTCP
7. **WAJIB** Relasi Lintas Materi
 - (a) **WAJIB** Pemilihan TCP vs UDP berdasarkan kebutuhan aplikasi
 - (b) **WAJIB** DNS dengan HTTP dan SMTP
 - (c) **WAJIB** Persistent HTTP dengan TCP Slow Start dan CongestionWindow
 - (d) **WAJIB** HTTPS, STARTTLS, dan SSH dengan Network Security
 - (e) **PAHAM** CDN sebagai routing tingkat aplikasi

7.2 Penjelasan Konsep

7.2.1 Fondasi Application Layer

Inti Konsep.

Application Layer adalah lapisan tempat proses aplikasi terdistribusi saling bertukar pesan untuk menjalankan fungsi seperti web, email, DNS, dan file transfer. Lapisan ini penting karena protokol aplikasi menentukan format pesan, makna field, tipe request/response, serta layanan transport yang dibutuhkan.

Application Layer berjalan pada ****end hosts****, bukan router inti. Proses aplikasi berada di ****user space****, yaitu ruang eksekusi aplikasi pengguna, sedangkan layanan transport seperti TCP/UDP disediakan oleh sistem operasi melalui socket. Router hanya meneruskan paket; ia tidak menjalankan logika aplikasi HTTP, SMTP, atau DNS sebagai bagian dari forwarding normal.

Sebuah protokol aplikasi mendefinisikan:

1. ****Sintaks****, yaitu format bit dan field pesan.
2. ****Semantik****, yaitu makna setiap field.
3. ****Tipe pesan****, misalnya request dan response.
4. ****Aturan interaksi****, yaitu kapan pesan dikirim dan bagaimana pihak lain harus merespons.

Arsitektur	Karakteristik
Client-server	Server selalu menyala, memiliki alamat IP relatif tetap, dan melayani banyak client. Client dapat terhubung intermiten dan biasanya tidak saling berkomunikasi langsung.
Peer-to-peer	Tidak ada server always-on; peer berkomunikasi langsung, sangat scalable, tetapi lebih sulit dikelola.
Hybrid	Menggabungkan server pusat untuk koordinasi dengan pertukaran data langsung antar-peer.

Tabel 11: Arsitektur aplikasi jaringan

Aplikasi memilih layanan berdasarkan kebutuhan. Transfer file dan email memerlukan no data loss, sehingga cocok memakai TCP. VoIP dan multimedia real-time lebih sensitif terhadap delay dan jitter, sehingga sering memakai UDP/RTP. Aplikasi elastic dapat memakai bandwidth berapa pun yang tersedia, sedangkan aplikasi multimedia sering membutuhkan minimum bandwidth atau delay bound.

Fondasi ini menghubungkan semua bagian Application Layer. DNS membantu aplikasi menemukan alamat, HTTP dan SMTP mendefinisikan format interaksi, sedangkan CDN/P2P mengatur distribusi konten pada skala besar.

7.2.2 Domain Name System (DNS)

Inti Konsep.

DNS adalah basis data terdistribusi hierarkis dan protokol Application Layer untuk menerjemahkan hostname menjadi alamat IP. DNS penting karena manusia memakai nama yang mudah diingat, sedangkan jaringan membutuhkan alamat IP yang dapat dirutekan.

Tanpa DNS, pengguna harus mengingat alamat IP numerik setiap layanan. Desain DNS tidak dibuat terpusat karena satu server akan menjadi single point of failure, tidak mampu menangani volume trafik global, jauh secara geografis, dan sulit dipelihara. Karena itu DNS memakai hierarki server dan caching.

Layanan DNS meliputi:

- **Host-to-IP translation**: menerjemahkan nama host menjadi IPv4/IPv6.
- **Host aliasing**: CNAME memetakan alias ke canonical name.
- **Mail aliasing**: MX record menunjuk mail server domain.
- **Load distribution**: DNS dapat mengembalikan beberapa IP dan merotasi urutannya.

Komponen DNS	Peran
Root name server	Mengetahui lokasi TLD server; secara logis sedikit tetapi direplikasi global.
TLD server	Mengelola domain seperti .com, .edu, .id, dan menunjuk authoritative server.
Authoritative name server	Menyimpan record resmi milik organisasi/domain.
Local resolver	Server DNS lokal ISP/kampus yang menerima kueri host, menelusuri hierarki, dan melakukan caching.

Tabel 12: Komponen hierarki DNS

Resolusi iterative:

1. Host bertanya ke local resolver.
2. Local resolver bertanya ke root.
3. Root memberi rujukan ke TLD.
4. TLD memberi rujukan ke authoritative server.
5. Authoritative server memberi record akhir.
6. Local resolver menyimpan jawaban dalam cache sampai TTL habis.

Resolusi recursive memindahkan tanggung jawab pencarian ke server yang ditanya, tetapi server tingkat atas umumnya menghindarinya karena beban besar. **TTL (Time-To-Live)** menentukan lama record boleh disimpan di cache. TTL panjang mengurangi trafik tetapi berisiko stale record; TTL pendek lebih segar tetapi membebani authoritative server.

Resource Record memakai format:

[Name, TTL, Class, Type, Value]

Type	Makna
A	Hostname ke alamat IPv4.
AAAA	Hostname ke alamat IPv6.
CNAME	Alias ke canonical name.
NS	Domain ke authoritative name server.
MX	Domain ke mail server.
PTR	Reverse lookup IP ke hostname.
TXT	Teks bebas untuk verifikasi dan kebijakan seperti SPF.

Tabel 13: Tipe Resource Record DNS

Perintah `nslookup` atau `dig` menunjukkan proses ini. Label ****Non-authoritative answer**** berarti jawaban berasal dari cache resolver lokal, bukan langsung dari authoritative server.

Komentar 7.1: Catatan: Sumber Pengetahuan Umum

DNSSEC ditambahkan dari pengetahuan umum dan RFC DNSSEC. DNSSEC memberi autentikasi asal data dan integritas record DNS menggunakan tanda tangan digital pada RRset. DNSSEC tidak memberi confidentiality; kueri dan jawaban DNS tetap dapat terlihat, tetapi penerima dapat memverifikasi bahwa data tidak dipalsukan selama resolusi.

DNS berhubungan langsung dengan HTTP dan SMTP. Browser perlu DNS sebelum membuka TCP ke server web, sedangkan SMTP memakai MX record untuk menemukan mail server domain tujuan.

7.2.3 Web dan HTTP

Inti Konsep.

HTTP adalah protokol Application Layer untuk Web yang bekerja dengan model client-server dan pull: browser meminta objek, server mengirim respons. HTTP penting karena ia mendefinisikan cara halaman web, gambar, script, cookie, cache, dan status respons dipertukarkan di atas TCP.

Sebuah halaman web terdiri dari banyak ****objects****. File ****base HTML**** berisi struktur halaman dan referensi ke objek lain seperti gambar, CSS, JavaScript, atau video. Setiap objek dialamati oleh ****URL (Uniform Resource Locator)**** yang mencakup hostname dan path.

HTTP berjalan di atas TCP, umumnya port 80 untuk HTTP biasa. HTTP tidak melakukan reliability sendiri; ia mengandalkan reliable byte stream TCP. HTTP bersifat ****stateless****, yaitu server tidak mengingat request sebelumnya sebagai bagian dari protokol dasar. Stateless memudahkan skalabilitas, tetapi fitur seperti login dan keranjang belanja membutuhkan mekanisme tambahan seperti cookies.

Non-persistent HTTP membuka koneksi TCP baru untuk setiap objek. Untuk satu objek:

$$\text{TotalTime} = 2\text{RTT} + \text{FileTransmissionTime}$$

Satu RTT dipakai untuk TCP three-way handshake; satu RTT berikutnya untuk request HTTP sampai byte pertama respons tiba; sisanya adalah waktu transmisi objek.

Persistent HTTP/1.1 menjaga koneksi TCP tetap terbuka dengan keep-alive. Pipelining memungkinkan client mengirim beberapa request berturut-turut tanpa menunggu respons satu per satu. Ini mengurangi biaya handshake dan membantu TCP CongestionWindow tidak terus kembali ke Slow Start.

Komponen HTTP	Definisi atau Peran
Request line	Method, path URL, dan versi HTTP.
Header	Metadata seperti <code>Host</code> , <code>User-Agent</code> , <code>Connection</code> , <code>Content-Type</code> .
Blank line	Penanda akhir header.
Entity body	Payload opsional, misalnya data form pada POST.
Status line	Versi HTTP, status code, dan reason phrase pada response.
Response body	Objek yang dikirim server, seperti HTML, JSON, atau gambar.

Tabel 14: Format pesan HTTP

Method penting: **GET** mengambil objek, **POST** mengirim data pada entity body, **PUT** mengunggah resource ke URL tertentu, dan **DELETE** menghapus resource. Status code penting: **200 OK**, **301 Moved Permanently**, **304 Not Modified**, dan **404 Not Found**.

Cookies membangun state di atas HTTP stateless:

1. Server mengirim header `Set-Cookie`.
2. Browser menyimpan cookie.
3. Request berikutnya membawa header `Cookie`.
4. Server memakai ID cookie untuk mengambil state dari database backend.

Web cache/proxy bertindak sebagai server bagi client lokal dan client bagi origin server. Cache mengurangi latency dan trafik access link. **Conditional GET** memvalidasi cache: proxy mengirim `If-Modified-Since` atau `If-None-Match`. Jika objek belum berubah, server membalas **304 Not Modified** tanpa body.

Komentar 7.2: Catatan: Sumber Pengetahuan Umum

HTTP/2 dan HTTP/3 ditambahkan dari pengetahuan umum dan standar modern. HTTP/2 memperkenalkan binary framing dan multiplexing beberapa stream HTTP di satu koneksi TCP, tetapi masih dapat mengalami head-of-line blocking pada level TCP. HTTP/3 menjalankan semantik HTTP di atas QUIC, sehingga multiplexing stream terjadi di atas UDP dengan recovery per-stream yang mengurangi dampak loss pada stream lain.

HTTP terkait langsung dengan Transport Layer dan Congestion Control. Non-persistent HTTP mengulang handshake dan Slow Start untuk setiap objek, sedangkan persistent HTTP membiarkan cwnd tumbuh dan membuat transfer banyak objek lebih efisien.

7.2.4 Electronic Mail

Inti Konsep.

Electronic Mail adalah aplikasi Application Layer asinkron untuk mengirim pesan melalui user agent, mail server, SMTP, dan mail access protocol. Email penting karena menunjukkan gabungan model push untuk pengiriman antar-server dan model pull untuk pengambilan pesan oleh pengguna.

Email membutuhkan reliability penuh: satu byte rusak atau hilang dapat mengubah isi pesan atau lampiran. Karena itu SMTP berjalan di atas TCP. Namun email tidak membutuhkan delay sangat kecil; pesan dapat antre di server ketika jaringan atau server tujuan belum siap.

Komponen	Peran
User Agent	Aplikasi untuk menulis, membaca, dan mengelola email.
Mail Server	Menyimpan mailbox dan meneruskan pesan antar-domain.
Mailbox	Kotak masuk pesan yang sudah tiba untuk pengguna.
Message queue	Antrean pesan keluar yang menunggu dikirim.
SMTP	Protokol push untuk memindahkan email antar-server dan dari sender ke server.

Tabel 15: Komponen sistem email

Alur email:

1. Alice menulis pesan di user agent.
2. User agent menyerahkan pesan ke mail server Alice.
3. Mail server Alice mencari MX record domain Bob melalui DNS.
4. SMTP client pada server Alice membuka TCP ke SMTP server tujuan pada port 25.
5. Server Bob menerima pesan dan menaruhnya di mailbox Bob.
6. Bob mengambil pesan menggunakan POP3, IMAP, atau webmail.

Dialog SMTP memakai teks ASCII:

Listing 5: Dialog SMTP ringkas

```

1 S: 220 hamburger.edu
2 C: HELO crepes.fr
3 S: 250 Hello crepes.fr
4 C: MAIL FROM:<alice@crepes.fr>
5 C: RCPT TO:<bob@hamburger.edu>
6 C: DATA
7 S: 354 Enter mail, end with "." on a line by itself
8 C: [message body]
9 C: .
10 C: QUIT
11 S: 221 closing connection

```

Format pesan dasar mengikuti RFC 822: header seperti **To**, **From**, **Subject**, lalu blank line, lalu body. Karena SMTP awalnya hanya membawa ASCII 7-bit, ****MIME**** menambahkan header seperti **MIME-Version**, **Content-Type**, dan **Content-Transfer-Encoding**. Base64 memetakan setiap 3 byte data biner menjadi 4 karakter ASCII. Multipart memakai boundary untuk memisahkan teks dan attachment.

POP3 memiliki fase authorization, transaction, dan update; modelnya sederhana dan dapat berupa download-and-delete. IMAP menyimpan pesan dan struktur folder di server, sehingga bersifat stateful dan cocok untuk banyak perangkat. Webmail memakai HTTP/HTTPS sebagai antarmuka pengguna, dengan cookies dan state server-side.

Email mengikat DNS, Transport, dan Security. DNS MX menemukan mail server tujuan; TCP memberi reliable transfer; STARTTLS dan PGP memberi keamanan untuk saluran atau isi pesan.

7.2.5 Arsitektur Lanjutan

Inti Konsep.

Arsitektur lanjutan seperti overlay, CDN, web services, dan P2P memperluas kemampuan Application Layer di atas Internet yang sudah ada. Konsep ini penting karena aplikasi modern sering mengatur routing, distribusi konten, dan integrasi layanan pada tingkat aplikasi.

****Overlay network**** adalah jaringan logis di atas jaringan fisik. Node overlay membuat keputusan forwarding sendiri menggunakan header logis, sedangkan router IP bawah hanya melihat header IP

luar. Overlay memungkinkan eksperimen routing, VPN, multicast logis, atau layanan baru tanpa mengubah router inti Internet.

CDN (Content Distribution Network) adalah kumpulan **surrogate servers** yang tersebar geografis untuk menyajikan konten lebih dekat ke pengguna. Motivasi CDN adalah menurunkan latency, menghindari congestion, mengurangi beban origin server, dan menghadapi flash crowd. CDN sering memakai **DNS redirection**: nama konten diarahkan ke server CDN yang paling cocok melalui CNAME atau jawaban DNS yang dipilih berdasarkan lokasi/beban.

Web Services memungkinkan aplikasi antar-mesin saling memanggil API. **REST** memakai resource yang diidentifikasi URI dan operasi HTTP seperti GET, POST, PUT, DELETE. **SOAP** memakai pesan terstruktur dengan envelope, header, dan body, biasanya berbasis XML.

P2P menghilangkan server always-on sebagai pusat data. **Gnutella** memakai unstructured P2P dan flooding query ke peer tetangga. **BitTorrent** memecah file besar menjadi **pieces**; peer mengunduh (**leeching**) dan mengunggah (**seeding**) potongan secara paralel, dibantu **tracker** untuk menemukan peer. **DHT (Distributed Hash Table)** adalah structured P2P yang memakai hashing objek ke ID numerik agar pencarian lebih terarah.

Arsitektur ini berkaitan dengan Network Layer karena CDN dan overlay dapat melakukan routing tingkat aplikasi yang berbeda dari jalur IP murni. Ia juga terkait dengan Congestion Control karena distribusi konten yang dekat dapat mengurangi beban bottleneck.

7.2.6 Protokol Aplikasi Khusus

Inti Konsep.

Protokol aplikasi khusus menunjukkan bahwa setiap aplikasi besar memiliki kebutuhan pesan dan transport yang berbeda. FTP membutuhkan reliability, SNMP membutuhkan manajemen ringan, sedangkan multimedia membutuhkan sesi, codec, port, timestamp, dan feedback real-time.

FTP (File Transfer Protocol) adalah protokol client-server untuk transfer file yang membutuhkan no data loss, sehingga berjalan di atas TCP. FTP cocok sebagai contoh aplikasi elastic: ia ingin throughput tinggi, tetapi tidak membutuhkan delay ketat seperti VoIP.

SNMP (Simple Network Management Protocol) digunakan untuk memantau dan mengelola perangkat jaringan. SNMP memakai UDP dan **MIB (Management Information Base)** sebagai abstraksi data manajemen. Operasi pentingnya:

- **GET**: mengambil nilai objek manajemen.
- **GETNEXT**: mengambil objek berikutnya dalam struktur MIB.
- **SET**: mengubah parameter perangkat.
- **TRAP**: notifikasi asinkron dari perangkat ke manajer.

Multimedia real-time memakai beberapa protokol. **SDP** mendeskripsikan sesi, codec, dan port. **SIP** menginisiasi sesi, menemukan pengguna, menegosiasikan media, dan dapat memakai proxy/forking. **H.323** adalah alternatif dari ITU-T dengan gatekeeper dan gateway. **RTP** membawa media di atas UDP dengan sequence number dan timestamp, sedangkan **RTCP** memberi feedback loss/delay.

Bagian ini menghubungkan Application Layer dengan Transport Layer. FTP memilih TCP karena reliability; SNMP dan RTP memilih UDP karena overhead rendah dan delay lebih penting.

7.2.7 Relasi Lintas Materi

Inti Konsep.

Application Layer selalu bergantung pada layer bawah untuk addressing, transport, routing, congestion behavior, dan security. Relasi ini penting karena desain aplikasi jaringan tidak bisa dipisahkan dari TCP/UDP, DNS, TLS, dan performa jaringan.

Pemilihan TCP vs UDP ditentukan oleh kebutuhan. HTTP, SMTP, dan FTP memakai TCP karena data harus lengkap dan berurutan. DNS memakai UDP untuk kueri pendek dan cepat, dengan TCP sebagai fallback untuk respons besar atau kebutuhan tertentu. RTP/VoIP memakai UDP karena retransmission TCP dapat membuat paket datang terlalu lambat.

DNS menjadi prasyarat banyak aplikasi. HTTP membutuhkan DNS untuk menerjemahkan hostname sebelum membuka koneksi TCP. SMTP membutuhkan MX record untuk menemukan mail server tujuan.

Persistent HTTP berhubungan dengan Congestion Control. Dengan mempertahankan satu koneksi TCP, HTTP menghindari pengulangan three-way handshake dan pengulangan Slow Start untuk setiap objek. Ini membuat Congestion Window dapat tumbuh lebih stabil.

Security muncul karena banyak protokol lama memakai plaintext. HTTPS membungkus HTTP dengan TLS, STARTTLS mengamankan SMTP, dan SSH menggantikan remote login plaintext. Pada CDN dan overlay, keputusan tingkat aplikasi dapat mengarahkan trafik ke server yang lebih dekat daripada sekadar mengikuti routing IP biasa.

7.3 Algoritma dan Rumus Penting

7.3.1 DNS Resource Record

Makna Rumus.

Format Resource Record menyatakan unit data dasar DNS. Rumus ini dipakai untuk memahami bagaimana hostname, TTL, tipe record, dan nilai jawaban disimpan dalam sistem DNS.

$$RR = [\text{Name}, \text{TTL}, \text{Class}, \text{Type}, \text{Value}] \quad (20)$$

dengan:

- Name: nama domain atau hostname.
- TTL: masa cache dalam detik.
- Class: kelas, biasanya IN untuk Internet.
- Type: jenis record seperti A, AAAA, CNAME, NS, atau MX.
- Value: isi record sesuai tipe.

7.3.2 Iterative DNS Resolution

Tujuan Algoritma.

Resolusi iteratif membuat local resolver menelusuri hierarki DNS dari root sampai authoritative server. Algoritma ini mengurangi beban rekursif pada server tingkat atas dan memungkinkan caching di resolver lokal.

Listing 6: Pseudocode iterative DNS resolution

```
1 Input: hostname
2 Output: resource records for hostname
3
4 resolver receives query from client
5 if cache has valid entry:
6     return cached answer
7
8 server = root_server
9 while server is not authoritative for hostname:
10     referral = ask(server, hostname)
11     server = referral.next_name_server
12
13 answer = ask(server, hostname)
```

```
14 store answer in cache until TTL expires
15 return answer to client
```

7.3.3 Non-persistent HTTP Response Time

Makna Rumus.

Rumus waktu respons non-persistent HTTP menunjukkan biaya satu koneksi TCP baru untuk satu objek. Rumus ini menjelaskan mengapa persistent HTTP jauh lebih efisien untuk halaman yang memiliki banyak objek.

$$\text{TotalTime} = 2\text{RTT} + \text{FileTransmissionTime} \quad (21)$$

dengan:

- RTT pertama: TCP three-way handshake.
- RTT kedua: request HTTP sampai byte pertama respons diterima.
- FileTransmissionTime: waktu mengirim seluruh objek.

7.3.4 Conditional GET

Tujuan Algoritma.

Conditional GET memvalidasi cache tanpa selalu mengunduh objek penuh. Jika salinan cache masih valid, server mengirim 304 Not Modified tanpa body.

Listing 7: Pseudocode Conditional GET

```
1 cache receives client request for object
2 if object is not cached:
3     fetch object from origin server
4     store object and validator
5 else:
6     send GET with If-Modified-Since or If-None-Match
7     if origin replies 304 Not Modified:
8         serve cached object
9     else:
10        replace cached object with new response
11        serve new object
```

7.3.5 Base64 Encoding Size

Makna Rumus.

Base64 dipakai MIME agar data biner dapat dibawa melalui channel email berbasis ASCII. Setiap 3 byte biner dipetakan menjadi 4 karakter ASCII, sehingga ukuran pesan bertambah.

$$3 \text{ byte biner} \rightarrow 4 \text{ karakter ASCII Base64} \quad (22)$$

Konsekuensinya, lampiran email yang dikodekan Base64 memiliki overhead ukuran sekitar sepertiga dibanding data biner asal, sebelum memperhitungkan header MIME dan boundary multipart.

8 Wireless Network

Wireless Network membahas jaringan yang memakai gelombang radio sebagai media transmisi. Materi ini penting karena sifat medium udara berbeda tajam dari kabel: sinyal melemah, dapat terinterferensi, memantul, mudah disadap, dan membuat packet loss yang tidak selalu berarti congestion.

Fokus ujian adalah memahami elemen jaringan nirkabel, mode infrastructure/ad hoc, standar Wi-Fi dan seluler, masalah link fisik, MAC 802.11 dengan CSMA/CA dan RTS/CTS, mobilitas, routing ad hoc, keamanan link-layer, serta dampaknya terhadap TCP dan security.

8.1 Outline Fundamental Konsep Materi

1. **WAJIB** Fondasi dan Elemen Wireless Network
 - (a) **WAJIB** Wireless host, base station, wireless link, dan wired infrastructure
 - (b) **WAJIB** Infrastructure mode, ad hoc mode, dan handoff
 - (c) **WAJIB** Taksonomi single-hop/multi-hop, infrastructure/no-infrastructure
2. **WAJIB** Standar dan Karakteristik Link
 - (a) **WAJIB** WLAN vs Wi-Fi
 - (b) **PAHAM** IEEE 802.15/Bluetooth dan IEEE 802.11b/a/g/n/ac/ax
 - (c) **PAHAM** Cellular 2G, 2.5G, 3G, 4G LTE/WiMAX, dan 5G[†]
 - (d) **WAJIB** Path loss, interference, dan multipath propagation
 - (e) **PAHAM** MIMO, frequency hopping, direct sequence, CDMA, dan chipping code
3. **WAJIB** MAC 802.11
 - (a) **WAJIB** Hidden terminal dan exposed terminal
 - (b) **WAJIB** Alasan CSMA/CD tidak cocok untuk wireless
 - (c) **WAJIB** CSMA/CA dan RTS/CTS/DATA/ACK
 - (d) **WAJIB** Scanning, beacon frame, authentication, dan association
 - (e) **PAHAM** Wi-Fi 6/802.11ax high-efficiency features[†]
4. **PAHAM** Mobilitas dan Routing
 - (a) **PAHAM** Mobile IP, home agent, foreign agent, dan tunneling
 - (b) **PAHAM** MACAW, AODV, OLSR, dan ExOR
5. **WAJIB** Wireless Security
 - (a) **WAJIB** Risiko medium broadcast
 - (b) **WAJIB** WEP, WPA2, IEEE 802.11i, dan CCMP[‡]
 - (c) **PAHAM** WPA3 dan SAE[†]
6. **WAJIB** Relasi Lintas Materi
 - (a) **WAJIB** TCP over wireless dan misinterpretasi loss
 - (b) **WAJIB** Ad hoc routing vs routing IP biasa
 - (c) **WAJIB** Kebutuhan link-layer security pada wireless

8.2 Penjelasan Konsep

8.2.1 Fondasi dan Elemen Wireless Network

Inti Konsep.

Wireless Network adalah jaringan yang memakai gelombang radio untuk mengirim data tanpa kabel fisik. Konsep ini penting karena konektivitas menjadi fleksibel dan mobile, tetapi medium udara membawa masalah baru seperti interferensi, atenuasi, handoff, dan keamanan siaran.

Motivasi utama wireless adalah mobilitas dan fleksibilitas. Laptop, smartphone, sensor, dan perangkat bergerak membutuhkan akses jaringan tanpa harus tersambung kabel. Namun, nirkabel tidak selalu berarti mobile: perangkat wireless dapat diam, dan perangkat mobile dapat berpindah antar-base station.

Elemen	Definisi atau Peran
Wireless host	End device seperti laptop atau smartphone yang menjalankan aplikasi dan memakai link radio.
Base station	Perangkat seperti access point atau cell tower yang menjadi relay antara host wireless dan jaringan kabel.
Wireless link	Medium udara/radio yang menghubungkan host dengan base station atau host lain.
Wired infrastructure	Infrastruktur kabel/Internet yang terhubung melalui base station.

Tabel 16: Elemen dasar wireless network

Mode operasi utama:

1. ****Infrastructure mode****: host berkomunikasi melalui base station yang terhubung ke jaringan kabel. Ketika host bergerak, ****handoff**** memindahkan koneksi dari satu base station ke base station lain.
2. ****Ad hoc mode****: tidak ada base station. Node berkomunikasi langsung jika berada dalam jangkauan dan dapat merutekan paket antar-node.

Taksonomi wireless network:

Kategori	Contoh / Makna
Single-hop infrastructure	Host langsung ke AP/cell tower, misalnya Wi-Fi atau cellular.
Single-hop no infrastructure	Komunikasi langsung tanpa Internet, misalnya Bluetooth sederhana.
Multi-hop infrastructure	Host mencapai base station melalui relay wireless lain, misalnya mesh network.
Multi-hop no infrastructure	Jaringan relay tanpa koneksi luar, misalnya MANET atau VANET.

Tabel 17: Taksonomi wireless network

Konsep ini berkaitan dengan Network Layer dan Transport Layer. Jika host berpindah jaringan, alamat dan rute dapat berubah; jika link radio menyebabkan loss, TCP dapat salah mengira loss tersebut sebagai congestion.

8.2.2 Standar dan Karakteristik Link

Inti Konsep.

Standar wireless menentukan jangkauan, frekuensi, modulasi, dan kapasitas transmisi; karakteristik link menentukan mengapa komunikasi radio lebih sulit daripada kabel. Bagian ini penting karena banyak masalah MAC, TCP, dan security berasal dari sifat fisik link wireless.

****WLAN**** adalah kategori jaringan lokal nirkabel. ****Wi-Fi**** adalah implementasi komersial WLAN berbasis standar IEEE 802.11 dan disertifikasi oleh Wi-Fi Alliance. Keduanya sering dipakai bergantian, tetapi secara teknis Wi-Fi adalah merek/ekosistem implementasi.

Standar	Karakteristik dari sumber
IEEE 802.15 / Bluetooth	Personal area network, indoor sekitar 10–30 m, sekitar 1 Mbps.
802.11b	2.4 GHz, sekitar 5–11 Mbps.
802.11a/g	Sekitar 54 Mbps; 802.11a di 5 GHz, 802.11g di 2.4 GHz.
802.11n	Wi-Fi 4, memakai MIMO, sekitar 450–600 Mbps.
802.11ac	Wi-Fi 5, 5 GHz, 256-QAM, theoretical rate hingga beberapa Gbps.
802.11ax	Wi-Fi 6, 2.4/5 GHz, 1024-QAM, theoretical rate hingga sekitar 9.6 Gbps.
2G/2.5G/3G/4G	Evolusi cellular dari puluhan Kbps sampai beberapa Mbps/Gbps bergantung standar dan lingkungan.

Tabel 18: Standar wireless yang disebut sumber

Tiga masalah fisik utama:

- **Path loss**: kekuatan sinyal melemah seiring jarak dan hambatan fisik.
- **Interference**: sinyal lain pada frekuensi yang sama/berdekatan merusak transmisi.
- **Multipath propagation**: sinyal memantul dan tiba lewat beberapa jalur dengan delay berbeda.

Teknik fisik untuk mengatasi keterbatasan ini meliputi **MIMO**, **frequency hopping**, **direct sequence spread spectrum**, dan **CDMA/chipping code**. MIMO memakai banyak antena untuk membuat spatial streams. Frequency hopping berpindah-pindah frekuensi. Direct sequence menyebar bit menjadi chip. CDMA memakai kode untuk memisahkan beberapa pengguna pada frekuensi yang sama.

Komentar 8.1: Catatan: Sumber Pengetahuan Umum

5G ditambahkan sebagai konteks modern dari evolusi cellular. Secara umum, 5G menargetkan throughput lebih tinggi, latency lebih rendah, dan dukungan perangkat masif dibanding generasi sebelumnya. Detail arsitektur 5G tidak menjadi fokus sumber NotebookLM untuk materi ini, sehingga catatan ini hanya menempatkannya sebagai kelanjutan standar cellular setelah 4G/LTE.

Karakteristik link ini menjelaskan mengapa MAC wireless berbeda dari Ethernet dan mengapa TCP dapat bermasalah di atas wireless.

8.2.3 MAC 802.11

Inti Konsep.

MAC 802.11 mengatur cara banyak host berbagi medium radio yang sama. Konsep ini penting karena wireless tidak dapat mengandalkan collision detection seperti Ethernet, sehingga harus memakai collision avoidance, RTS/CTS, ACK link-layer, scanning, dan association.

Masalah MAC wireless berbeda dari kabel karena setiap node tidak selalu mendengar node lain. **Hidden terminal problem** terjadi ketika A dan C tidak saling mendengar, tetapi keduanya mengirim ke B sehingga tabrakan terjadi di B. **Exposed terminal problem** terjadi ketika node menahan transmisi karena mendengar tetangga, padahal transmisinya ke penerima lain tidak akan mengganggu.

CSMA/CD pada Ethernet tidak cocok untuk wireless karena:

1. Pengirim sulit mendeteksi collision sambil memancarkan sinyal sendiri.
2. Sinyal yang diterima dapat sangat lemah akibat fading dan path loss.
3. Collision dapat terjadi di penerima walaupun tidak terdeteksi di pengirim, terutama pada hidden terminal.

Karena itu 802.11 memakai **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance). Intinya, host mendengar channel sebelum mengirim dan berusaha menghindari collision sebelum terjadi.

Prosedur RTS/CTS:

1. Sender mengirim **RTS** (Request to Send) ke receiver.
2. Receiver membalas **CTS** (Clear to Send).
3. Node yang mendengar CTS diam selama durasi transmisi agar hidden terminal tidak mengganggu.
4. Sender mengirim DATA.
5. Receiver mengirim ACK link-layer jika frame diterima utuh.

Prosedur bergabung ke AP:

1. AP menyiarkan **beacon frame** yang berisi identitas jaringan.
2. Client melakukan **scanning** untuk menemukan AP.
3. Client melakukan **authentication**, misalnya verifikasi kredensial/kunci.
4. Client melakukan **association** agar AP menerima dan meneruskan frame miliknya ke wired infrastructure.

Komentar 8.2: Catatan: Sumber Pengetahuan Umum

Fitur high-efficiency Wi-Fi 6/802.11ax ditambahkan dari sumber standar/umum. Contohnya OFDMA untuk membagi channel ke resource unit kecil, MU-MIMO untuk melayani banyak perangkat, BSS coloring untuk mengurangi gangguan antar-BSS, Target Wake Time untuk efisiensi daya, dan spatial reuse untuk meningkatkan kapasitas area padat. Sumber NotebookLM menyebut 802.11ax dan MIMO, tetapi tidak merinci semua fitur ini.

MAC 802.11 berkaitan langsung dengan reliability lokal. ACK di link-layer tidak menggantikan TCP ACK end-to-end, tetapi membantu mendeteksi kegagalan frame pada hop wireless.

8.2.4 Mobilitas dan Routing

Inti Konsep.

Mobilitas dan routing wireless menjaga konektivitas ketika host bergerak atau ketika tidak ada base station pusat. Konsep ini penting karena routing IP biasa mengasumsikan topologi relatif stabil, sedangkan wireless ad hoc dan mobile host dapat berubah posisi dan rutenya.

Pada infrastructure mode, mobilitas link-layer ditangani dengan handoff antar-AP. Namun, jika host berpindah subnet IP, koneksi transport dapat terganggu karena alamat IP berubah. **Mobile IP** mencoba mempertahankan identitas IP host.

Mekanisme Mobile IP:

1. Host memiliki jaringan asal dengan **home agent**.
2. Saat berpindah ke jaringan lain, host berhubungan dengan **foreign agent**.
3. Paket yang dikirim ke alamat asli host dicegat oleh home agent.
4. Home agent melakukan **tunneling**, yaitu membungkus paket dan mengirimkannya ke foreign agent.
5. Foreign agent membongkar paket dan menyerahkannya ke mobile host.

Pada ad hoc multi-hop, tidak ada AP yang menjadi pusat routing. Node harus menemukan dan memelihara rute sendiri. Sumber menyebut beberapa pendekatan:

Protokol	Peran
MACAW	Perbaikan akses medium wireless untuk mengurangi hidden/exposed terminal.
AODV	Reactive ad hoc routing; mencari rute saat dibutuhkan.
OLSR	Proactive link-state routing; menjaga peta topologi secara berkala.
ExOR	Opportunistic routing yang memanfaatkan peluang beberapa node penerima.

Tabel 19: Contoh routing dan MAC lanjutan pada wireless

Mobilitas menghubungkan Wireless Network dengan Network Layer dan Transport Layer. Perubahan rute, subnet, dan kualitas link dapat mengubah RTT, loss, dan koneksi TCP yang sedang berjalan.

8.2.5 Wireless Security

Inti Konsep.

Wireless security diperlukan karena radio disiarkan ke udara dan dapat didengar oleh perangkat dalam jangkauan. Keamanan wireless harus bekerja sejak link-layer agar frame tidak mudah disadap, dipalsukan, atau dimodifikasi sebelum mencapai lapisan transport/aplikasi.

Berbeda dari kabel, penyerang wireless tidak harus menyambung fisik ke medium. Cukup berada dalam jangkauan sinyal, penyerang dapat mengamati trafik. Karena itu wireless memerlukan autentikasi dan enkripsi link-layer.

Mekanisme	Makna
WEP	Upaya awal memberi privasi setara kabel, tetapi lemah dan mudah diretas.
WPA2 / IEEE 802.11i	Pengamanan modern yang menggantikan WEP dan memakai mekanisme kunci serta AES.
CCMP	Protokol berbasis AES untuk confidentiality dan integrity frame MAC.

Tabel 20: Evolusi keamanan wireless

Komentar 8.3: Catatan: Perbedaan dengan Sumber Lain

NotebookLM menyebut CCMP sebagai CTR Counter Mode. Penjelasan yang lebih tepat dari referensi keamanan 802.11 adalah CCMP berbasis AES-CCM, yaitu kombinasi counter mode untuk enkripsi dan CBC-MAC/MIC untuk autentikasi serta integritas. Jadi, CCMP bukan sekadar CTR; ia adalah skema authenticate-and-encrypt untuk frame 802.11.

Komentar 8.4: Catatan: Sumber Pengetahuan Umum

WPA3 dan SAE ditambahkan sebagai konteks modern. WPA3-Personal memakai SAE (Simultaneous Authentication of Equals) untuk menggantikan PSK handshake lama dan memperkuat perlindungan terhadap serangan tebakan password offline. Detail WPA3 tidak dibahas sebagai topik utama sumber NotebookLM, sehingga catatan ini hanya menempatkannya sebagai evolusi setelah WPA2.

Wireless security berkaitan dengan Network Security. TLS atau IPsec tetap berguna, tetapi wireless juga membutuhkan perlindungan Data Link karena medium udara terbuka bahkan sebelum paket mencapai layer atas.

8.2.6 Relasi Lintas Materi

Inti Konsep.

Wireless Network memengaruhi layer lain karena packet loss, mobilitas, dan broadcast medium mengubah asumsi jaringan kabel. Relasi paling penting adalah TCP over wireless: loss radio dapat disalahartikan sebagai congestion.

Dengan **Transport Layer**, masalah utama adalah interpretasi loss. TCP klasik menganggap packet loss sebagai tanda congestion. Pada wireless, loss dapat terjadi karena path loss, interference, atau multipath, bukan karena router bottleneck. Akibatnya TCP melakukan multiplicative decrease secara sia-sia, cwnd turun, dan throughput merosot.

Dengan **Network Layer**, ad hoc mode memerlukan routing yang berbeda dari routing IP biasa. Topologi dapat berubah cepat, node dapat menjadi relay, dan rute harus ditemukan atau dipelihara secara dinamis.

Dengan **Network Security**, wireless membutuhkan link-layer security karena medium siar terbuka. Firewall di jaringan kabel tidak cukup untuk mencegah penyadapan radio; frame harus dilindungi sejak hop wireless.

Dengan **Congestion Control**, loss wireless menantang asumsi bahwa loss berarti queue overflow. Ini menjelaskan mengapa desain lintas-layer atau sinyal eksplisit dapat membantu membedakan congestion loss dari wireless error loss.

8.3 Algoritma dan Rumus Penting

8.3.1 CSMA/CA dengan RTS/CTS

Tujuan Algoritma.

CSMA/CA menghindari collision sebelum terjadi karena wireless sulit melakukan collision detection. RTS/CTS membantu mengurangi hidden terminal dengan membuat node sekitar receiver ikut diam.

Listing 8: Pseudocode CSMA/CA dengan RTS/CTS

```
1 Input: frame to send, destination
2 Output: frame delivered or retransmission attempted
3
4 while frame not delivered and retry_limit not exceeded:
5     sense_channel()
6     if channel_is_busy:
7         wait_random_backoff()
8         continue
9
10    send_RTS(destination)
11    if CTS_received:
12        send_DATA(destination)
13        if ACK_received:
14            mark_delivered()
15        else:
16            wait_random_backoff()
17    else:
18        wait_random_backoff()
```

8.3.2 Mobile IP Tunneling

Tujuan Algoritma.

Mobile IP tunneling mempertahankan konektivitas host yang bergerak ke jaringan asing. Home agent membungkus paket yang menuju alamat permanen host dan mengirimkannya ke foreign agent.

Listing 9: Pseudocode Mobile IP tunneling

```
1 correspondent sends packet to mobile_host_home_address
2 home_agent intercepts packet
3 home_agent encapsulates packet to foreign_agent_address
4 foreign_agent decapsulates packet
5 foreign_agent delivers original packet to mobile_host
```

8.3.3 Delay Spread Intuition pada Multipath

Makna Rumus.

Multipath membuat salinan sinyal tiba pada waktu berbeda. Selisih waktu kedatangan ini membantu menjelaskan mengapa simbol dapat saling mengganggu pada link wireless.

$$\Delta t = t_{\text{late}} - t_{\text{early}} \quad (23)$$

dengan:

- t_{late} : waktu kedatangan salinan sinyal yang lebih lambat.
- t_{early} : waktu kedatangan salinan sinyal yang lebih cepat.

Komentar 8.5: Keterbatasan Sumber

Sumber utama menjelaskan multipath propagation secara konseptual, tetapi tidak memberikan model matematis kanal radio yang rinci. Persamaan delay spread di atas adalah abstraksi dasar untuk membantu memahami dampak multipath, bukan rumus performa lengkap dari sumber.

9 Network Security

Network Security membahas cara membangun komunikasi aman di atas Internet yang pada dasarnya terbuka, digunakan bersama, dan tidak seluruh infrastrukturnya dapat dipercaya. Fokus utama materi ini adalah memahami tujuan keamanan, ancaman per-layer, blok kriptografi, autentikasi, distribusi kunci, protokol keamanan, firewall, dan serangan availability.

Materi ini menutup rangkaian UAS karena seluruh layer sebelumnya dapat menjadi target serangan. IP dapat dipalsukan, TCP dapat diserang SYN flood atau hijacking, aplikasi dapat memiliki bug, wireless dapat disadap, dan congestion dapat dipicu secara sengaja oleh DDoS.

9.1 Outline Fundamental Konsep Materi

1. **WAJIB** Fondasi Network Security
 - (a) **WAJIB** Shared resource, untrusted infrastructure, Alice, Bob, Mallory
 - (b) **WAJIB** Authentication, confidentiality, integrity, availability
2. **WAJIB** Kerentanan per Layer
 - (a) **WAJIB** IP spoofing, broadcast amplification, Smurf attack, routing attacks
 - (b) **WAJIB** TCP SYN flood, session hijacking, dan session reset
 - (c) **WAJIB** Malware, buffer overflow, dan application-layer attacks
3. **WAJIB** Cryptographic Building Blocks
 - (a) **WAJIB** Plaintext, ciphertext, encryption, decryption, dan key
 - (b) **WAJIB** Symmetric cipher, stream cipher, block cipher, CBC, DES, 3DES, AES
 - (c) **WAJIB** Public-key cipher dan RSA
 - (d) **WAJIB** Hash, message digest, collision resistance, one-way property, uniformity
 - (e) **WAJIB** MAC, HMAC, nonce, dan challenge-response
4. **WAJIB** Key Predistribution, PKI, dan Authentication Protocols
 - (a) **WAJIB** KDC, session key, CA, X.509, PKI, CRL, chain of trust
 - (b) **PAHAM** Web of trust dan PGP
 - (c) **WAJIB** Diffie-Hellman, MitM, Needham-Schroeder, dan Kerberos
5. **WAJIB** Security Systems by Layer
 - (a) **WAJIB** PGP, SSH, TLS/SSL/HTTPS, IPsec, dan wireless security
 - (b) **PAHAM** TLS 1.3[†]
 - (c) **WAJIB** CCMP pada WPA2/802.11i[‡]
6. **WAJIB** Firewalls, Filtering, Malware, dan Availability Attacks
 - (a) **WAJIB** Stateful packet filter, proxy firewall, DMZ, ingress/egress filtering
 - (b) **PAHAM** NAT sebagai perlindungan topologi internal
 - (c) **WAJIB** Worm, virus, botnet, DoS, dan DDoS
 - (d) **PAHAM** DNSSEC[†]
7. **WAJIB** Relasi Lintas Materi
 - (a) **WAJIB** HTTP plaintext menjadi HTTPS dengan TLS
 - (b) **WAJIB** IP tanpa autentikasi memerlukan IPsec untuk proteksi network-layer
 - (c) **WAJIB** Wireless membutuhkan link-layer encryption
 - (d) **WAJIB** DDoS memengaruhi resource allocation dan congestion control

9.2 Penjelasan Konsep

9.2.1 Fondasi Network Security

Inti Konsep.

Network Security bertujuan membuat komunikasi tetap aman walaupun jaringan adalah shared resource dan melewati infrastruktur yang tidak seluruhnya dipercaya. Konsep ini penting karena penyerang dapat menyadap, memalsukan, mengubah, atau melumpuhkan komunikasi di berbagai layer.

Internet adalah **shared resource**, yaitu sumber daya bersama yang dipakai oleh banyak pihak dengan kepentingan berbeda. Sumber NotebookLM menekankan bahwa cryptography adalah ilmu komunikasi di hadapan musuh. Model klasiknya memakai **Alice** sebagai pengirim sah, **Bob** sebagai penerima sah, dan **Mallory** sebagai penyerang yang dapat menyadap, memodifikasi, atau menyisipkan pesan.

Masalah utama keamanan jaringan adalah **untrusted infrastructure**. Pesan dari Alice ke Bob dapat melewati ISP, router, dan jaringan antara yang tidak dikendalikan Alice atau Bob. Karena itu, protokol keamanan harus tetap benar walaupun jalur di tengah tidak dipercaya.

Tujuan Keamanan	Makna
Authentication	Memastikan identitas lawan komunikasi benar.
Confidentiality	Menjaga isi data tidak dapat dibaca pihak tak berwenang.
Integrity	Menjamin data tidak diubah, ditambah, atau dikurangi tanpa terdeteksi.
Availability	Menjaga layanan tetap dapat diakses walaupun ada serangan.

Tabel 21: Tujuan utama secure communication channel

Keempat tujuan ini muncul kembali pada semua protokol keamanan. TLS menargetkan authentication, confidentiality, dan integrity untuk aplikasi; firewall dan mitigasi DDoS lebih dekat ke availability; WPA2/802.11i memberi proteksi link-layer untuk medium wireless.

9.2.2 Kerentanan per Layer

Inti Konsep.

Setiap layer jaringan memiliki asumsi yang dapat dieksploitasi. IP dapat dipalsukan, routing dapat diblokkan, TCP dapat dibanjiri atau dibajak, dan aplikasi dapat memiliki bug seperti buffer overflow.

Pada **Network Layer**, IP secara dasar mempercayai source address yang ditulis host pengirim. Ini membuka **IP spoofing**, yaitu pemalsuan alamat sumber. Spoofing dapat dipakai untuk menipu sistem yang memakai alamat IP sebagai autentikasi atau untuk melakukan amplification attack.

Smurf attack adalah contoh broadcast amplification. Penyerang mengirim ICMP Echo Request ke broadcast network dengan source IP dipalsukan menjadi IP korban. Banyak host di broadcast network membalas ke korban, sehingga trafik ke korban berlipat ganda.

Routing juga rentan. Pada BGP, penyerang dapat melakukan prefix hijacking atau path alteration. Pada OSPF, penyerang dapat menyebarkan informasi link-state palsu. Pada RIP, penyerang dapat mengumumkan rute dengan metrik sangat rendah agar trafik mengarah padanya.

Pada **Transport Layer**, serangan umum meliputi:

Serangan	Mekanisme
TCP SYN flood	Penyerang mengirim banyak SYN sehingga server mengalokasikan state setengah terbuka sampai resource habis.
Session hijacking	Penyerang menebak sequence number dan menyisipkan paket untuk mengambil alih koneksi sah.
Session reset	Penyerang mengirim paket RST palsu untuk memutus koneksi.

Tabel 22: Serangan Transport Layer

Pada **Application Layer**, serangan sering menargetkan bug program, malware, worm, virus, atau buffer overflow. Jika input tidak divalidasi, penyerang dapat menimpa memori dan mengambil alih eksekusi program.

Kerentanan per-layer menunjukkan bahwa keamanan tidak bisa hanya dipasang di satu tempat. Aplikasi, transport, network, dan data link membutuhkan perlindungan sesuai ancumannya masing-masing.

9.2.3 Cryptographic Building Blocks

Inti Konsep.

Cryptographic building blocks adalah alat matematika dasar untuk confidentiality, integrity, dan authentication. Blok ini meliputi cipher simetris, cipher asimetris, hash, MAC/HMAC, dan nonce.

Encryption mengubah **plaintext** menjadi **ciphertext** menggunakan **key**. **Decryption** membalik ciphertext menjadi plaintext. Tanpa key yang benar, ciphertext harus tampak seperti data acak bagi penyerang.

Jenis	Makna
Symmetric-key cipher	Alice dan Bob memakai shared secret yang sama; cepat dan cocok untuk data besar.
Stream cipher	Menghasilkan keystream pseudo-random lalu di-XOR dengan plaintext.
Block cipher	Memecah data ke blok tetap dan mengenkripsi tiap blok, misalnya DES, 3DES, AES.
Public-key cipher	Memakai pasangan public key/private key; lebih lambat tetapi membantu distribusi kunci.

Tabel 23: Klasifikasi cipher

Pada stream cipher, intuisi operasinya:

$$C = P \oplus K_s$$

dengan P plaintext, K_s keystream, dan C ciphertext. Penerima menghasilkan keystream yang sama lalu melakukan XOR lagi untuk memperoleh plaintext.

Pada block cipher, mode **CBC (Cipher Block Chaining)** mengaitkan blok:

$$C_i = E_K(P_i \oplus C_{i-1})$$

dengan P_i blok plaintext, C_i blok ciphertext, E_K fungsi enkripsi dengan key K , dan C_{i-1} ciphertext blok sebelumnya. Tujuannya agar blok plaintext identik tidak langsung menghasilkan ciphertext identik.

Public-key cryptography memakai public key untuk operasi yang boleh diketahui umum dan private key untuk rahasia pemiliknya. RSA adalah contoh utama yang disebut sumber. Karena lebih lambat daripada symmetric cipher, public-key cipher sering dipakai untuk autentikasi atau pertukaran key, sedangkan data besar dienkripsi dengan session key simetris.

Cryptographic hash function memetakan pesan panjang sembarang menjadi **message digest** berukuran tetap. Sifat yang diperlukan:

- **One-way**: sulit mencari pesan asal dari digest.
- **Collision resistant**: sulit mencari dua pesan berbeda dengan digest sama.
- **Uniformity**: keluaran tersebar merata agar tidak mudah dieksploitasi.

Hash tanpa key tidak cukup untuk autentikasi karena Mallory dapat mengubah pesan dan menghitung hash baru. **MAC/HMAC** menambahkan shared key:

$$\text{HMAC} = H(K \parallel M)$$

secara konseptual, dengan K key rahasia dan M pesan. Bob menghitung ulang HMAC dengan key yang sama. Jika cocok, pesan dianggap berasal dari pihak yang tahu key dan tidak berubah.

Untuk mencegah replay, protokol memakai **nonce**, yaitu random bitstring yang hanya dipakai sekali. Pada challenge-response, Alice mengirim nonce; Bob membalas MAC atas nonce tersebut. Mallory tidak dapat memakai ulang respons lama karena nonce berubah.

9.2.4 Key Predistribution, PKI, dan Authentication Protocols

Inti Konsep.

Distribusi kunci dan autentikasi menjawab masalah: bagaimana dua pihak yang belum pernah bertemu dapat memperoleh key yang benar dan yakin identitas lawannya sah. Konsep ini penting karena cipher kuat tetap gagal jika key diberikan kepada pihak yang salah.

Komentar 9.1: Keterbatasan Sumber

Query mendalam NotebookLM untuk PKI, TLS, IPsec, firewall, dan malware beberapa kali mengembalikan jawaban yang tidak sesuai konteks. Bagian ini disusun dari Outline Final NotebookLM yang sudah disetujui dan pengetahuan umum standar keamanan jaringan, bukan dari jawaban mendalam yang berhasil untuk subtopik ini.

Pada sistem symmetric-key, **Key Distribution Center (KDC)** adalah pihak tepercaya yang membantu dua pihak memperoleh **session key**. Session key dipakai untuk satu sesi komunikasi agar key jangka panjang tidak langsung dipakai untuk seluruh trafik.

Pada public-key, masalahnya adalah memastikan public key benar-benar milik pihak yang diklaim. **Certificate Authority (CA)** menandatangani **X.509 certificate** yang mengikat identitas dengan public key. **PKI (Public Key Infrastructure)** mencakup CA, certificate, **CRL (Certificate Revocation List)**, dan **chain of trust** dari root CA ke intermediate CA hingga certificate server.

Web of trust adalah alternatif desentralisasi yang dipakai dalam ekosistem seperti PGP: pengguna saling menandatangani key, bukan bergantung penuh pada CA pusat.

Diffie-Hellman key agreement memungkinkan dua pihak membangun shared secret melalui kanal terbuka. Namun, Diffie-Hellman murni rentan **Man-in-the-Middle** jika tidak diautentikasi: Mallory dapat membuat key terpisah dengan Alice dan Bob sambil berada di tengah.

Needham-Schroeder dan **Kerberos** adalah contoh protokol autentikasi berbasis pihak tepercaya. Kerberos memakai ticket dan timestamp untuk mengurangi risiko replay serta memudahkan autentikasi dalam sistem client/server.

Semua mekanisme ini mendukung protokol layer atas. TLS memakai certificate dan key exchange; PGP memakai kombinasi public-key dan symmetric-key; Kerberos menyediakan autentikasi terpusat dalam domain administrasi.

9.2.5 Security Systems by Layer

Inti Konsep.

Security systems dapat ditempatkan di berbagai layer: PGP di aplikasi, TLS di atas transport, IPsec di network layer, dan WPA2/802.11i di data link wireless. Posisi layer menentukan apa yang dilindungi dan siapa yang harus mendukung protokol tersebut.

PGP mengamankan email pada Application Layer. Pola umumnya: pesan dienkripsi dengan symmetric session key karena efisien, lalu session key tersebut dienkripsi dengan public key penerima. PGP juga dapat memakai tanda tangan digital dan web of trust.

SSH menggantikan remote login plaintext. Sumber outline membagi SSH menjadi:

- **SSH-TRANS**: membangun terowongan transport terenkripsi.
- **SSH-AUTH**: mengautentikasi user ke server.
- **SSH-CONN**: memultipleksikan channel di atas koneksi aman.

TLS/SSL/HTTPS mengamankan aplikasi seperti web di atas TCP. Handshake menegosiasikan cipher suite, memvalidasi certificate server, membentuk pre-master secret, lalu menurunkan session keys. Setelah handshake, **record protocol** memecah data aplikasi menjadi record, menambahkan proteksi integrity, lalu mengenkripsi record.

Komentar 9.2: Catatan: Sumber Pengetahuan Umum

TLS 1.3 ditambahkan sebagai konteks modern. TLS 1.3 menyederhanakan handshake, menghapus banyak algoritma lama, memakai AEAD cipher suites, dan menekankan forward secrecy melalui ephemeral Diffie-Hellman. Mode 0-RTT dapat mempercepat koneksi ulang, tetapi memiliki pertimbangan replay risk sehingga tidak cocok untuk semua jenis request.

IPsec bekerja pada Network Layer. **Transport mode** melindungi payload IP antar-host tanpa membungkus seluruh paket IP lama; **tunnel mode** membungkus paket IP penuh dan umum dipakai VPN antar-gateway. **AH (Authentication Header)** memberi integrity dan anti-replay tanpa confidentiality. **ESP (Encapsulating Security Payload)** memberi encryption dan dapat juga memberi integrity. **IKE** menegosiasikan key dan security association.

Wireless security bekerja pada Data Link Layer. WEP sudah usang. WPA2/IEEE 802.11i memakai mekanisme kunci dan AES.

Komentar 9.3: Catatan: Perbedaan dengan Sumber Lain

Outline NotebookLM menyebut CCMP sebagai CTR Counter Mode. Penjelasan yang lebih tepat: CCMP berbasis AES-CCM, yaitu counter mode untuk enkripsi dan CBC-MAC/MIC untuk autentikasi serta integritas frame. Jadi, CCMP bukan hanya enkripsi CTR, tetapi skema authenticate-and-encrypt untuk 802.11i/WPA2.

Pemilihan layer menentukan cakupan. TLS melindungi aplikasi tertentu, IPsec dapat melindungi semua IP traffic antar-host/gateway, sedangkan WPA2 hanya melindungi hop wireless.

9.2.6 Firewalls, Filtering, Malware, dan Availability Attacks

Inti Konsep.

Firewall dan filtering membatasi trafik yang boleh masuk atau keluar, sedangkan malware, botnet, DoS, dan DDoS menyerang availability. Konsep ini penting karena confidentiality dan integrity tidak cukup jika layanan dapat dibuat tidak tersedia.

Komentar 9.4: Keterbatasan Sumber

Query mendalam NotebookLM untuk firewall dan malware juga mengembalikan jawaban HTTP yang tidak relevan. Isi berikut mengikuti Outline Final NotebookLM yang sudah disetujui dan pengetahuan umum standar dari materi keamanan jaringan.

Firewall adalah titik kontrol perimeter yang menyaring trafik. **Stateful packet filter** melacak state koneksi, misalnya apakah paket TCP merupakan bagian dari koneksi yang sudah sah. **Proxy firewall** bertindak sebagai perantara aplikasi, menerima koneksi dari client dan membuat

koneksi baru ke server. **DMZ (Demilitarized Zone)** adalah zona untuk server publik agar tidak langsung berada di jaringan internal sensitif.

Ingress filtering menolak paket masuk yang tampak memalsukan alamat internal. **Egress filtering** membatasi paket keluar agar host internal yang terinfeksi tidak mudah dipakai untuk spoofing atau DDoS keluar. **NAT** menyembunyikan alamat private internal dan membuat koneksi masuk langsung lebih sulit, tetapi NAT bukan pengganti firewall penuh.

Malware mencakup virus dan worm. Virus biasanya membutuhkan program host atau aksi pengguna; worm menyebar otomatis melalui jaringan dengan mengeksploitasi vulnerability, termasuk zero-day. Banyak host terinfeksi dapat menjadi **botnet**, yaitu kumpulan mesin yang dikendalikan command-and-control untuk spam, scanning, atau DDoS.

DoS (Denial of Service) berusaha membuat layanan tidak tersedia dengan menghabiskan bandwidth, CPU, memory, socket, atau state server. **DDoS** melakukan hal yang sama dari banyak sumber sekaligus, sering memakai botnet.

DDoS berhubungan dengan Congestion Control karena trafik palsu memenuhi link dan buffer, memicu packet drop dan delay tinggi. Bagi flow sah, dampaknya tampak seperti congestion berat walaupun penyebabnya adalah serangan.

Komentar 9.5: Catatan: Sumber Pengetahuan Umum

DNSSEC ditambahkan sebagai contoh keamanan infrastruktur aplikasi. DNSSEC memberi autentikasi asal data dan integritas record DNS melalui tanda tangan digital, tetapi tidak menyembunyikan isi query atau jawaban. Ini berbeda dari TLS yang memberi confidentiality untuk koneksi aplikasi.

9.2.7 Relasi Lintas Materi

Inti Konsep.

Network Security mengikat seluruh materi jaringan: aplikasi butuh TLS, IP butuh mekanisme autentikasi tambahan, wireless butuh link-layer encryption, dan DDoS dapat merusak congestion control. Keamanan harus dipahami sebagai properti lintas-layer, bukan fitur satu protokol saja.

Dengan **Application Layer**, HTTP plaintext mudah disadap dan dimodifikasi, sehingga HTTPS memakai TLS. SMTP dapat diamankan dengan STARTTLS, sedangkan email end-to-end dapat memakai PGP.

Dengan **Transport Layer**, TCP memberi reliability tetapi tidak otomatis memberi confidentiality atau authentication. TLS ditempatkan di atas TCP untuk melindungi byte stream aplikasi.

Dengan **Network Layer**, IP tidak memvalidasi source address dan routing protocol dapat dibajak. IPsec menambahkan authentication, integrity, dan encryption pada level IP.

Dengan **Wireless Network**, medium radio terbuka bagi penyadap sehingga security harus ada sejak link-layer. WPA2/802.11i/CCMP melindungi frame wireless, sedangkan TLS tetap diperlukan untuk end-to-end security aplikasi.

Dengan **Congestion Control**, DDoS dan worm traffic dapat menciptakan congestion buatan. Serangan availability tidak hanya merusak server, tetapi juga menghabiskan bandwidth dan buffer jaringan.

9.3 Algoritma dan Rumus Penting

9.3.1 Stream Cipher XOR

Makna Rumus.

Stream cipher mengenkripsi dengan menggabungkan plaintext dan keystream melalui XOR. Rumus ini menunjukkan mengapa pengirim dan penerima harus menghasilkan keystream yang sama dari key yang sama.

$$C = P \oplus K_s \quad (24)$$

$$P = C \oplus K_s \quad (25)$$

dengan P plaintext, C ciphertext, dan K_s keystream.

9.3.2 Cipher Block Chaining

Makna Rumus.

CBC mengaitkan blok plaintext dengan ciphertext blok sebelumnya agar pola plaintext identik tidak langsung terlihat sebagai pola ciphertext identik.

$$C_i = E_K(P_i \oplus C_{i-1}) \quad (26)$$

dengan:

- P_i : plaintext block ke- i .
- C_i : ciphertext block ke- i .
- E_K : enkripsi block cipher dengan key K .
- C_{i-1} : ciphertext block sebelumnya.

9.3.3 HMAC Konseptual

Makna Rumus.

HMAC menggabungkan message dan secret key agar integrity sekaligus authentication dapat diverifikasi. Penyerang yang tidak tahu key tidak dapat membuat HMAC valid untuk pesan yang diubah.

$$\text{HMAC} = H(K \parallel M) \quad (27)$$

dengan H fungsi hash, K shared secret key, M pesan, dan $\parallel$ operasi konkatenasi. Bentuk ini adalah intuisi konseptual; konstruksi HMAC nyata memakai inner/outer padding.

9.3.4 Challenge-Response dengan Nonce

Tujuan Algoritma.

Challenge-response dengan nonce membuktikan bahwa pihak lawan mengetahui key saat ini, bukan sekadar memutar ulang jawaban lama.

Listing 10: Pseudocode challenge-response

```

1 Alice -> Bob: nonce N
2 Bob computes R = HMAC(K_AB, N)
3 Bob -> Alice: R
4 Alice computes R_expected = HMAC(K_AB, N)
5 if R == R_expected:
6     Bob is authenticated for this fresh challenge
7 else:
8     reject

```

9.3.5 SYN Flood Intuition

Tujuan Algoritma.

SYN flood mengeksploitasi state yang dibuat server saat menerima SYN. Intuisi ini menjelaskan mengapa serangan availability dapat terjadi bahkan sebelum aplikasi menerima request penuh.

Listing 11: Intuisi SYN flood

```
1 attacker repeats many times:
2     send TCP SYN with spoofed or unreachable source address
3
4 server for each SYN:
5     allocate half-open connection state
6     send SYN-ACK
7     wait for final ACK that never arrives
8
9 result:
10    backlog/state table fills
11    legitimate clients cannot connect
```